



MODBUS: спецификация протокола приложения

Добавил(а) microsin

MODBUS это протокол обмена сообщениями уровня приложения, позиционируемый как уровень 7 модели OSI (Open Systems Interconnection). Modbus предоставляет способ обмена данными типа клиент/сервер между устройствами, подключенных к различным типам шин или сетей. Непонятные термины и сокращения см. в разделе "Словарик", в конце статьи.

В индустрии широко применяются в основном 2 варианта соединения устройств - последовательный (с 1979 года, на основе RS-232), и через сеть стека TCP/IP, через порт 502.

Протокол MODBUS работает по принципу запрос/ответ, и предоставляет службы, определенных функциональными кодами. Функциональные коды MODBUS это элементы запроса/ответа PDU (Protocol Data Unit). Назначение этого документа (перевод спецификации [1]) - описать функциональные коды, используемые в рабочей среде транзакций MODBUS.

Как уже упоминалось, MODBUS работает на уровне приложений, для обмена по типу клиент/сервер между устройствами, подключенными к разным типам шин или сетей. В настоящий момент реализованы следующие виды сетей:

- TCP/IP over Ethernet [2].
- Асинхронное последовательное соединение через различные физические реализации (проводные: EIA/TIA-232-E, EIA-422, EIA/TIA-485-A; оптика, радио, и т. д.).
- MODBUS PLUS, высокоскоростная сеть с передачей маркеров (high speed token passing network).

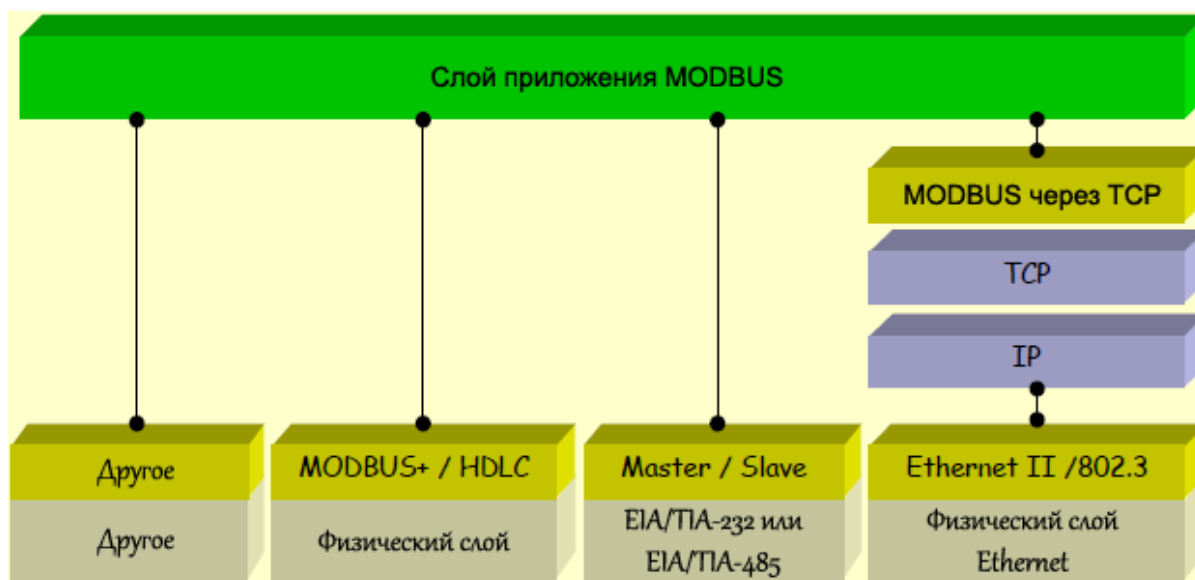


Рис. 1. Стек коммуникаций MODBUS.

Протокол MODBUS позволяет упростить обмен между всеми архитектурами сетей различных типов.

Коммуникации MODBUS

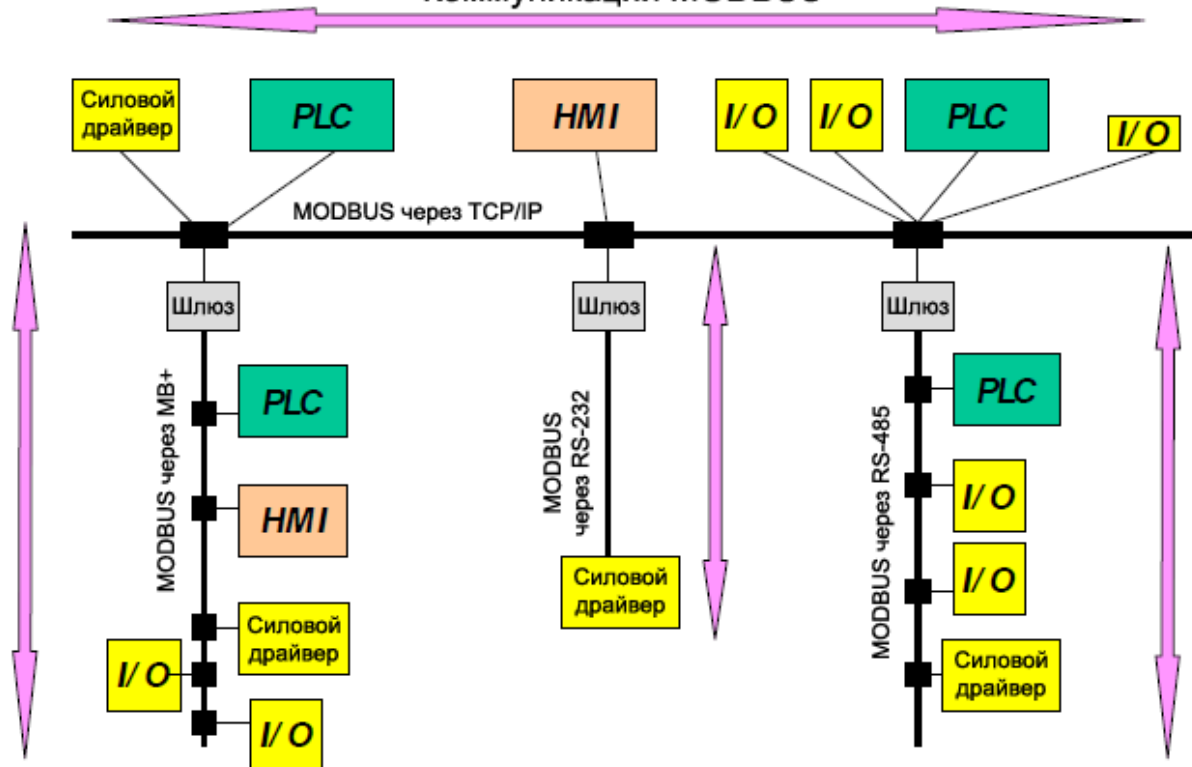


Рис. 2: Пример архитектуры сети MODBUS.

Каждый тип устройств (PLC, HMI, панель управления, драйвер, система управления движением, устройство I/O, ...) может использовать протокол MODBUS для инициирования сетевых (дистанционных) операций.

Один и тот же обмен данными может быть реализован как через последовательную линию связи (RS-232 и его производные), так и через сети Ethernet TCP/IP. Шлюзы позволяют осуществить коммуникацию между различными типами шин или сетей, используя протокол MODBUS.

[4. Общее описание протокола]

В протоколе MODBUS определена простая единица обмена данными (Protocol Data Unit, PDU), которая не зависит от нижележащих слоев обмена данными. Отображение протокола MODBUS на определенные шины или сети могут вводить дополнительные поля на единицу обмена данных приложения (Application Data Unit, ADU).

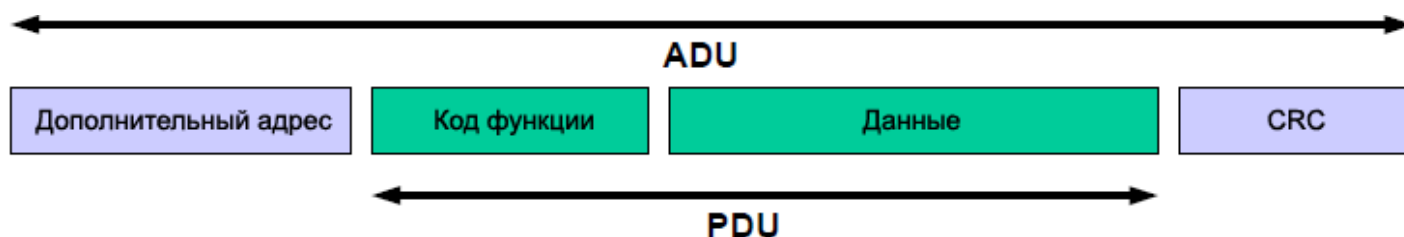


Рис. 3. Основной фрейм MODBUS.

MODBUS ADU формируется клиентом, который инициирует транзакцию MODBUS. Функция указывает серверу, какое действие следует выполнить. Протокол приложения MODBUS устанавливает формат запроса, инициированного клиентом.

Поле кода функции элемента данных MODBUS кодируется в одном байте. Допустимы коды в диапазоне 1 .. 255 (диапазон 128 .. 255 зарезервирован, и используется для ответов на исключения). Когда сообщение отправляется от клиента к серверу, поле кода функции говорит серверу, какое действие нужно предпринять. Код функции 0 недопустим.

Для некоторых кодов функции добавлены субкоды, чтобы определить несколько действий.

Поле данных сообщений отправляемое от клиента к серверу, содержит дополнительную информацию, которую сервер использует для выполнения действия, определенного кодом функции. Это может включать элементы наподобие дискретных

значений и адресов регистров, количества обрабатываемых элементов и количества фактических байт данных в поле.

Поле данных может отсутствовать (иметь нулевую длину) в определенных видах запросов - в этом случае сервер не требует дополнительную информацию. Код функции сам по себе определяет действие.

Если не было ошибки, связанной с запрошенной функцией MODBUS в корректно принятом поле данных MODBUS ADU, ответ сервера клиенту будет содержать запрошенные данные. Если произошла ошибка, связанная запрошенной функцией MODBUS, поле содержит код исключения, который приложение сервера может использовать, чтобы определить следующее предпринимаемое действие.

Например, клиент может прочитать состояния ON/OFF группы дискретных выходов или входов, или может прочитать/записать содержимое данных группы регистров.

Когда сервер отвечает клиенту, он использует поле кода функции, чтобы показать нормальный (безошибочный) ответ, или некоторый вид произошедшей ошибки (это называется ответ исключения, exception response). Для нормального ответа сервер просто как эхо выдает в ответ на запрос оригинальный код функции.

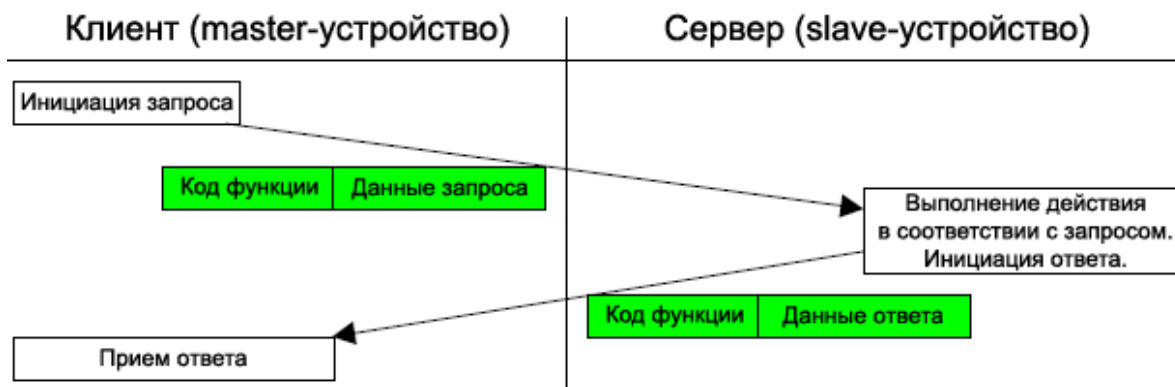


Рис. 4. Транзакция MODBUS (без ошибки).

Для ответа исключения сервер вернет код, эквивалентный оригинальному коду функции из запроса PDU, но у которого старший бит установлен в лог. 1.

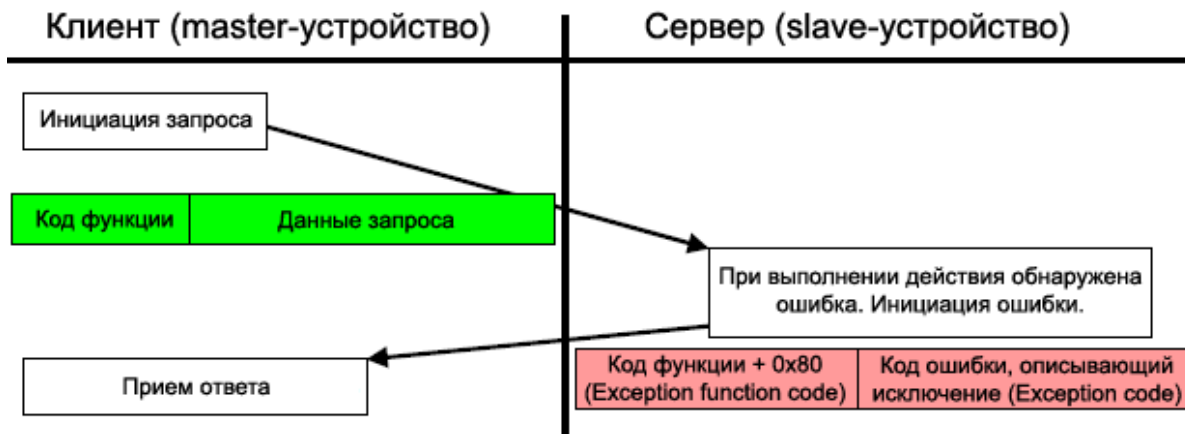


Рис. 5. Транзакция MODBUS (exception response, в случае ошибки).

Примечание: желательно отслеживать тайм-аут ответа, чтобы не ждать бесконечно ответа, который возможно никогда не придет.

Размер MODBUS PDU ограничен значением, унаследованным от реализации MODBUS на основе проводной последовательной сети (максимум RS485 ADU = 256 байт).

Таким образом, MODBUS PDU для последовательной линии обмена = 256 - адрес сервера (1 байт) - CRC (2 байта) = 253 байта.

Следовательно:

RS232 / RS485 ADU = 253 байта + адрес сервера (1 байт) + CRC (2 байта) = 256 байт.

TCP MODBUS ADU = 253 байта + MBAP (7 байт) = 260 байт.

Протокол MODBUS определяет три вида PDU:

- MODBUS Request PDU (PDU запроса), [mb_req_pdu](#).
- MODBUS Response PDU (PDU ответа), [mb_rsp_pdu](#).
- MODBUS Exception Response PDU (PDU ответа исключения), [mb_excep_rsp_pdu](#).

Определение запроса `mb_req_pdu`

`mb_req_pdu` определен следующим образом:

`mb_req_pdu = {function_code, request_data}`, где:

`function_code` = [1 байт] код функции MODBUS

`request_data` = [n байт] это поле зависит от кода функции, и обычно содержит информацию, такую как ссылки на переменные, значения счетчиков, смещения данных, коды субфункций и т. п.

Определение нормального ответа `mb_rsp_pdu`

`mb_rsp_pdu` определен следующим образом:

`mb_rsp_pdu = {function_code, response_data}`, где:

`function_code` = [1 байт] код функции MODBUS

`request_data` = [n байт] это поле зависит от кода функции, и обычно содержит информацию, такую как ссылки на переменные, значения счетчиков, смещения данных, коды субфункций и т. п.

Определение ответа исключения `mb_excep_rsp_pdu`

`mb_excep_rsp_pdu` определен следующим образом:

`mb_excep_rsp_pdu = {exception-function_code, request_data}`, где

`exception-function_code` = [1 байт] код функции MODBUS + 0x80

`exception_code` = [1 байт] код исключения (MODBUS Exception Code), определенный в таблице кодов исключений MODBUS (см. секцию 7 в конце статьи).

4.2. Кодирование данных. MODBUS использует big-Endian представление для адресов и элементов данных. Это значит, что если числовое значение использует для кодирования и передачи больше одного байта, то тогда сначала передается самый старший байт (в памяти самый старший байт размещается первым, т. е. по самому младшему адресу). Например, если размер регистра 16 бит, и в нем находится значение 0x1234, то сначала передается байт 0x12, затем 0x34. Подробнее см. [3, 4].

4.3. Модель данных MODBUS. MODBUS базируется на модели данных из серии таблиц, обладающих разными характеристиками. 4 главные таблицы следующие:

Основные таблицы	Тип объекта	Режим доступа	Комментарии
Discretes Input (цифровые входы)	Одиночный бит	Read-Only (только чтение)	Этот тип данных может быть предоставлен подсистемой ввода/вывода.
Coils ("обмотки", цифровые выходы)	Одиночный бит	Read-Write (чтение и запись)	Этот тип данных программа приложения может изменять.
Input Registers (входные регистры)	16-битное слово	Read-Only (только чтение)	Этот тип данных может быть предоставлен подсистемой ввода/вывода.
Holding Registers (регистры хранения)	16-битное слово	Read-Write (чтение и запись)	Этот тип данных программа приложения может изменять.

Различия между входами и выходами, а также между адресуемыми побитно и адресуемыми словами элементами данных не предполагают какого-либо поведения приложения. Вполне приемлемо и очень распространено рассмотрение всех четырех таблиц как перекрывающиеся друг друга, если это самая естественная интерпретация целевой машины.

Для каждой из главных таблиц протокол позволяет делать индивидуальный выбор их 65536 элементов данных. Также разработаны операции чтения или записи этих элементов данных как несколько следующих друг за другом элементов данных, вплоть до предела размера данных, который зависит от кода функции транзакции.

Очевидно, что все данные, обрабатываемые через MODBUS (биты, регистры) должны находиться в памяти приложения устройства. Однако физическое размещение в памяти (адрес) не следует путать со ссылкой на данные. The only requirement is to link data reference with physical address.

Логические ссылочные номера MODBUS, которые используются в функциях MODBUS, это беззнаковые целочисленные индексы (uint16_t), начинающиеся с 0.

Примеры реализации модели MODBUS

Примеры ниже показывают два способа организации данных в устройстве. Возможны различные организации, однако не все они описаны в этом документе. Каждое устройство имеет собственную организацию в соответствии с требованиями приложения.

[Пример 1: в устройстве 4 отдельных блока данных]

Этот пример показывает организацию данных в устройстве, у которых есть цифровые и аналоговые входы и выходы. Каждый блок отделен от других, потому что данные в разных блоках не связаны друг с другом. Таким образом, каждый блок доступен с помощью разных функций MODBUS.

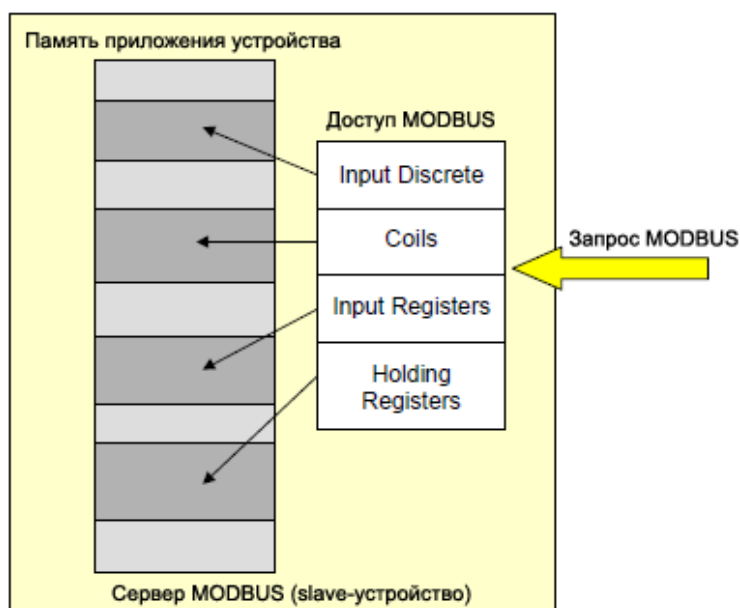
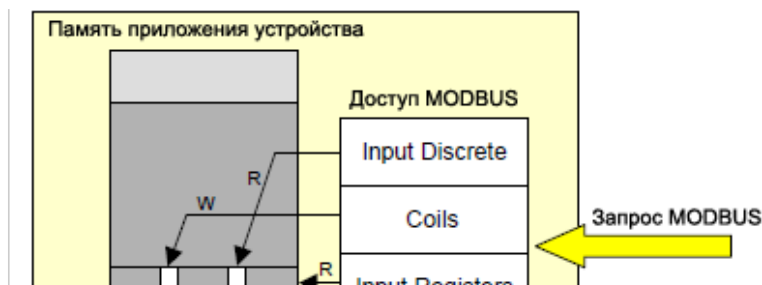


Рис. 6. Модель данных MODBUS с отдельными блоками.

[Пример 2: в устройстве только один блок данных]

В этом примере одни и те же данные могут быть доступны через разные функции MODBUS, либо через 16-битный доступ, либо через одноразрядный.



Модель адресации MODBUS. Протокол приложения MODBUS точно определяет правила адресации PDU. В MODBUS PDU каждые данные адресуются индексом от 0 до 65535.

Также ясно определена модель данных MODBUS, составленная из 4 блоков, каждый из которых пронумерован от 1 до n. Модели данных MODBUS каждый элемент в блоке нумеруется от 1 до n.

После этого модель данных MODBUS должна быть связана с приложением устройства (объект IEC-61131, или другая модель приложения). Предварительное отображение между моделью данных MODBUS и приложением устройства полностью определяется его производителем, и зависит от специфики устройства.

На рисунке ниже показано, как данные MODBUS с номером X адресованы в MODBUS PDU X-1.

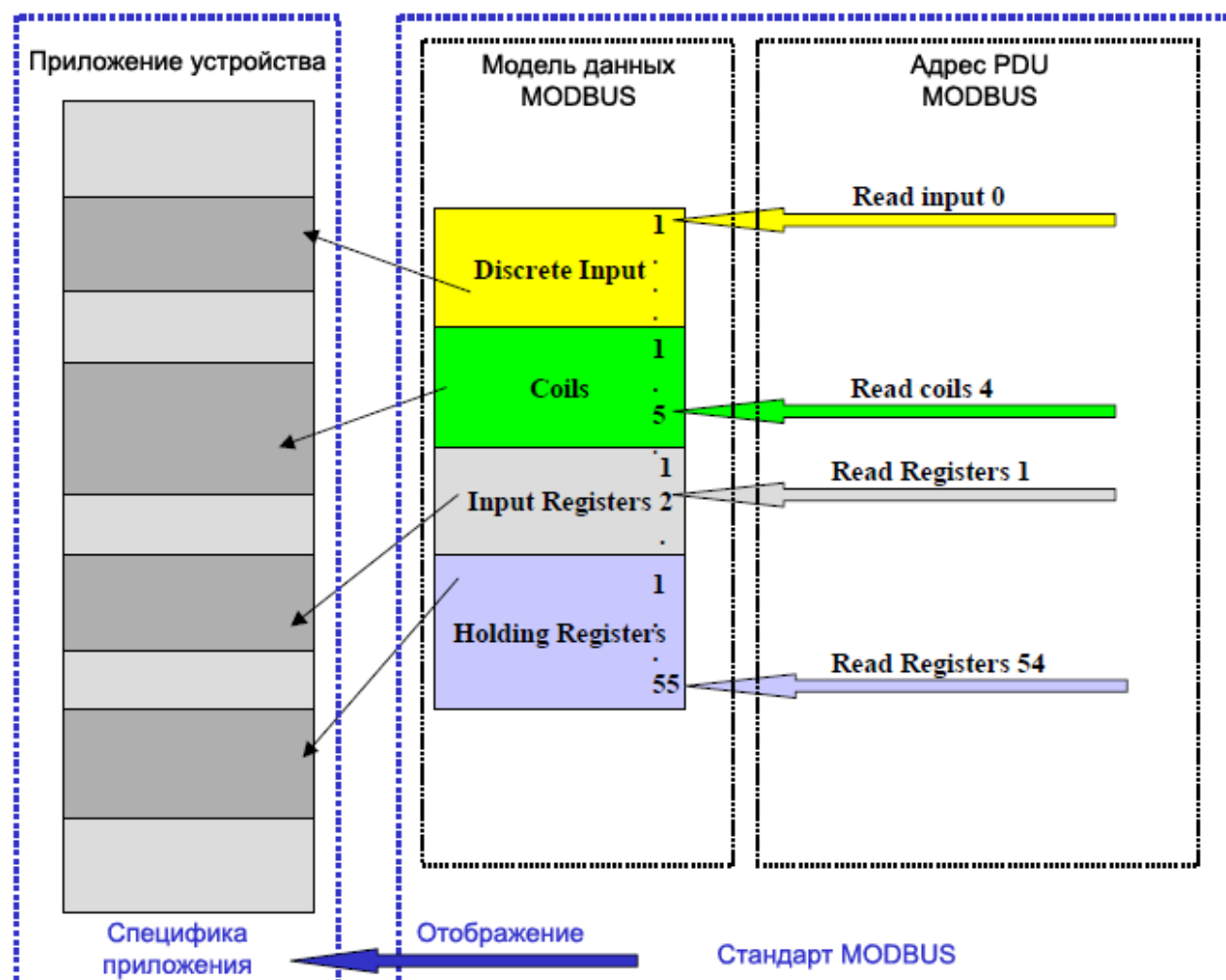


Рис. 8. Модель адресации MODBUS.

Транзакция MODBUS. Следующая диаграмма состояний описывает традиционную обработку транзакции MODBUS на стороне сервера.

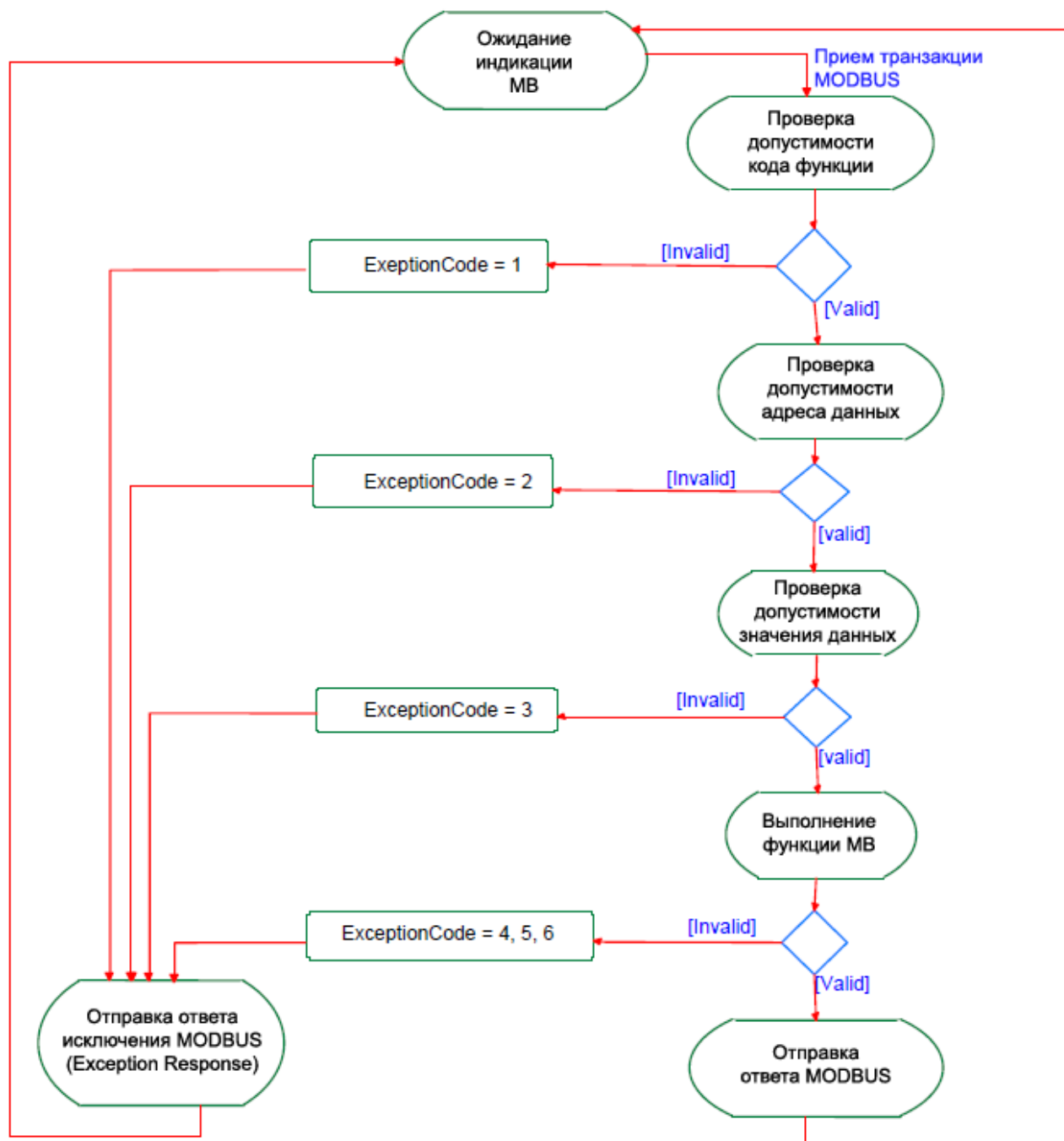


Рис. 9. Диаграмма состояний транзакции MODBUS.

Как только запрос был обработан сервером, он строит ответ с адекватной транзакцией MODBUS. В зависимости от результата обработки будет собран один из двух типов ответа:

- Положительный ответ MODBUS:
 - Код функции ответа = коду функции запроса.
- Ответ исключения (MODBUS Exception response, см. секцию 7):
 - Цель ответа предоставить клиенту подходящую информацию, описывающую ошибку, детектированную при обработке запроса.
 - Код исключения (exception function code) = коду функции запроса + 0x80;
 - Предоставляет код исключения, показывающий причину ошибки.

[5. Категории кода функции]

Существует 3 категории кодов функции MODBUS:

Публичные коды (Public Function Codes)

- Коды функций, которые хорошо определены.
- Гарантированно уникальные.
- Поддерживаются комьюнити MODBUS-IDA.org.
- Документация опубликована.
- Доступен тест совместимости.
- Включают как публично назначенные коды функций, так и не назначенные коды, зарезервированные для использования в будущем.

Пользовательские коды (User-Defined Function Codes)

- Два диапазона определяемых пользователем кодов, от 65 до 72 и от 100 до 110.
- Пользователь может выбрать и реализовать код функции, который не поддерживается спецификацией.
- Нет гарантии, что выбранный код функции будет уникальным.
- Если пользователь захочет сменить позиционирование функционала своего кода функции как публичного, он должен инициировать RFC, чтобы внести изменение в публичную категорию, в которой будет назначен новый функциональный код.
- Организация MODBUS, Inc явно оставляет за собой право разработки предлагаемого RFC.

Зарезервированные коды (Reserved Function Codes)

- Коды функции, используемые в настоящее время некоторыми компаниями в своих старых продуктах. Эти коды недоступны для публичного использования.

Дополнительно см. Приложение А "Зарезервированные коды MODBUS, субкоды и типы MEI" в конце этой документации.

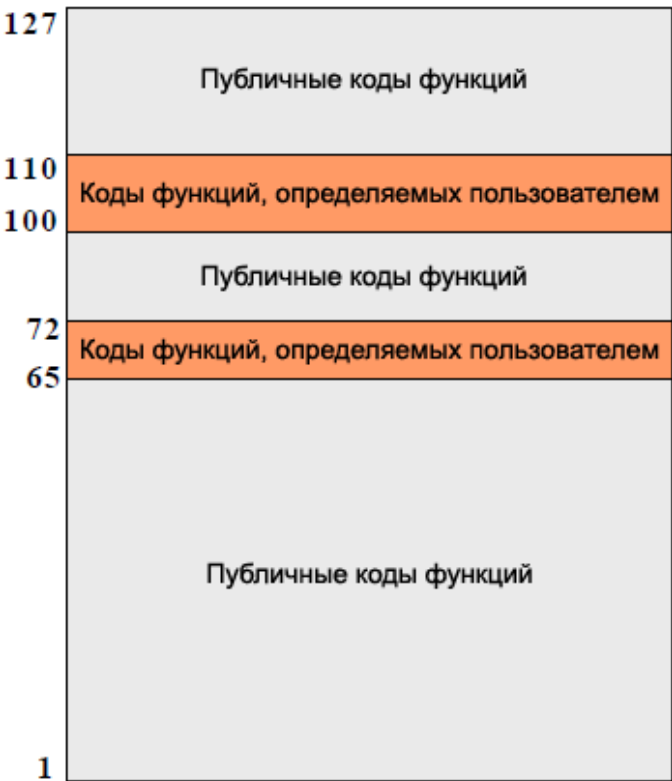


Рис. 10. Категории кодов функции.

Публичные коды функций:

		Функция	Коды функций		HEX	Секция
			Код	Субкод		
Данные		Read Coils	01		01	6.1
	Доступ	Физические дискретные входы, внутренние биты	02		02	6.2
	к битам	или физические выходы ("обмотки", coils)	05		05	6.5
		Write Multiple Coils	15		0F	6.11

	Физические входные регистры	Read Input Registers	04	04	6.4
		Read Holding Registers	03	03	6.3
		Write Single Register	06	06	6.6
		Write Multiple Registers	16	10	6.12
		Read/Write Multiple Registers	23	17	6.17
		Mask Write Register	22	16	6.16
16-битный доступ	Внутренние регистры или физические выходные регистры	Read FIFO queue	24	18	6.18
		Read File record	20	14	6.14
		Write File record	21	15	6.15
		Read Exception status	07	07	6.7
		Diagnostic	08	00-18, 20	6.8
		Get Com event counter	11	0B	6.9
Диагностика	Доступ к элементам файла	Get Com Event Log	12	0C	6.10
		Report Slave ID	17	11	6.13
		Read device Identification	43	14	6.21
Другое		Encapsulated Interface Transport	43	13, 14	6.19

[6. Описания кодов функции]

6.1. 01 (0x01) Read Coils

Этот код функции используется для чтения от 1 до 2000 смежных состояний обмоток в сетевом устройстве. Запрос PDU задает начальный адрес, т. е. адрес первой обмотки, и количество обмоток. В PDU обмотки адресуются от нуля. Таким образом, обмотки с номерами 1 .. 16 адресуются как 0 .. 15.

Обмотки в сообщении ответа упакованы как один бит поля данных на обмотку. Состояние включено (ON) определяется лог. 1 бита и выключено (OFF) лог. 0. LSB первого байта данных содержит выход, адресованный в запросе. Другие обмотки следуют по битам дальше, к более старшим битам до конца байта, и снова начинаются с младшего бита дальше, в последующих байтах.

Если возвращаемое количество выходов не делится нацело на 8, то оставшиеся биты последнего байта данных будут дополнены нулями (в направлении к старшим битам, до конца байта). Поле количества байт (Byte Count) указывает количество полных байт данных.

Запрос:

Код функции	1 байт	0x01
Начальный адрес	2 байта	0x0000 .. 0xFFFF
Количество выходов (coils)	2 байта	1 .. 2000 (0x0000 .. 0x07D0)

Ответ:

Код функции	1 байт	0x01
Количество байт	1 байт	N*
Статус обмоток	n байт	n = N или N+1

Примечание *: N = количество выходов / 8, и если остаток не равен 0, то N = N+1.

Ошибка:

Код функции	1 байт	Оригинальный код функции + 0x80
-------------	--------	--

Код исключения (exception code)

1 байт

Один из кодов от 01 до 04

Пример запроса на чтение дискретных выходов 20 .. 38**Запрос**

Имя поля

Функция

Старший байт адреса

Младший байт адреса

Старший байт количества выходов

Младший байт количества выходов

Ответ

Имя поля

Функция

Количество байт

Статус выходов 27..20

Статус выходов 35..28

Статус выходов 38..36

HEX

HEX

HEX

HEX

HEX

HEX

Статус выходов 27 .. 20 показан как значение байта 0xCD (hex), или бинарное значение 11001101. Выход 27 находится в MSB этого байта, и выход 20 в LSB.

По принятому соглашению биты в байте показаны с MSB слева, а LSB справа. Таким образом, выходы в первом байте от 27 до 20, слева направо. Следующий байт данных содержит выходы от 35 до 28, слева направо. Поскольку биты передаются последовательно, то их поток передает биты от LSB до MSB: 20, ..., 27, 28, ..., 35, и так далее.

В последнем байте данных статус выходов 38 .. 36 показан как байт со значением 0x05 (hex), или в двоичном виде 00000101. Выход 38 соответствует шестая позиции бита слева, и выход 36 соответствует LSB в этом байте. 5 оставшихся более

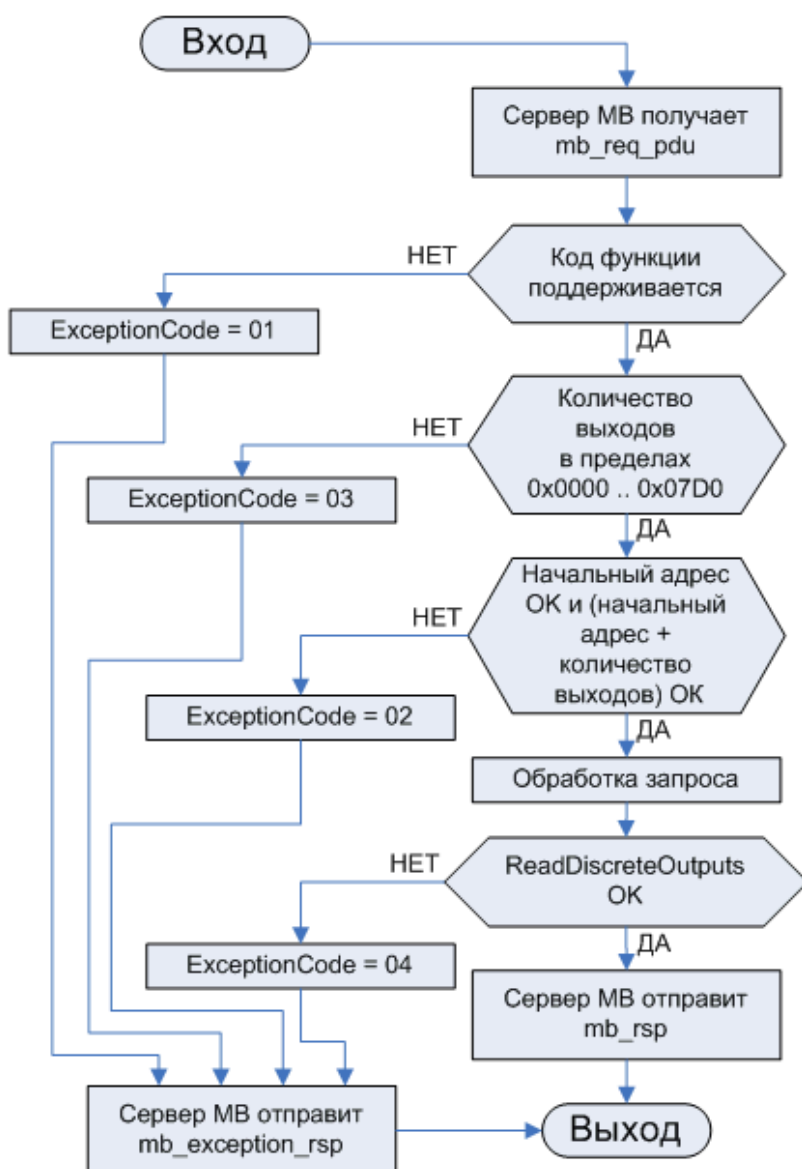


Рис. 11. Диаграмма состояний Read Coils.

6.2. 02 (0x02) Read Discrete Inputs

Этот код функции используется для чтения от 1 до 2000 смежных дискретных входов сетевого устройства. Запрос PDU задает начальный адрес, т. е. адрес первого заданного входа, и количество входов. В PDU Discrete Inputs адресация начинается с 0. Таким образом дискретные входы, пронумерованные 1 .. 16, адресуются как 0 .. 15.

Дискретные входы в сообщении ответа упакованы по одному входу на бит в поле данных. Состояние ON показывается как лог. 1, OFF как лог. 0. LSB первого байта данных содержит вход первый вход, адресованный в запросе. Другие входы помещаются в более старших битах, вплоть до конца байта, и снова начинаются с младших бит следующих байт.

Если возвращаемое количество входов не делится нацело на 8, то оставшиеся биты последнего байта данных будут дополнены нулями (в направлении к старшим битам, до конца байта). Поле количества байт (Byte Count) указывает количество полных байт данных.

Запрос:

Код функции	1 байт	0x02
Начальный адрес	2 байта	0x0000 .. 0xFFFF
Количество входов	2 байта	1 .. 2000 (0x0001 .. 0x07D0)

Ответ:

Код функции	1 байт	0x02
Количество байт	1 байт	N*
Статус входов	N* байт	

Примечание *: N = Количество входов / 8, и если остаток отличается от 0, то N = N+1.

Ошибка:

Код функции	1 байт	0x82
Код исключения (exception code)	1 байт	Один из кодов от 01 до 04

Пример запроса на чтение дискретных входов 197 .. 218

Запрос		Ответ	
Имя поля	HEX	Имя поля	HEX
Функция	02	Функция	02
Старший байт адреса	00	Количество байт	03
Младший байт адреса	C4	Статус входов 204..197	AC
Старший байт количества входов	00	Статус входов 212..205	DB
Младший байт количества входов	16	Статус входов 218..213	35
Статус дискретных входов 204 .. 197 показан как байт со значением 0xAC (hex), или двоичное значение 10101100. Вход 204 соответствует MSB в этом байте, а 197 соответствует LSB.			
Статус дискретных входов 218 .. 213 показан как байт со значением 0x35 (hex), или двоичное значение 00110101. Вход 218 соответствует третьей позиции бита слева и вход 213 соответствует LSB. Оставшиеся не используемые биты второго байта заполнены нулями.			

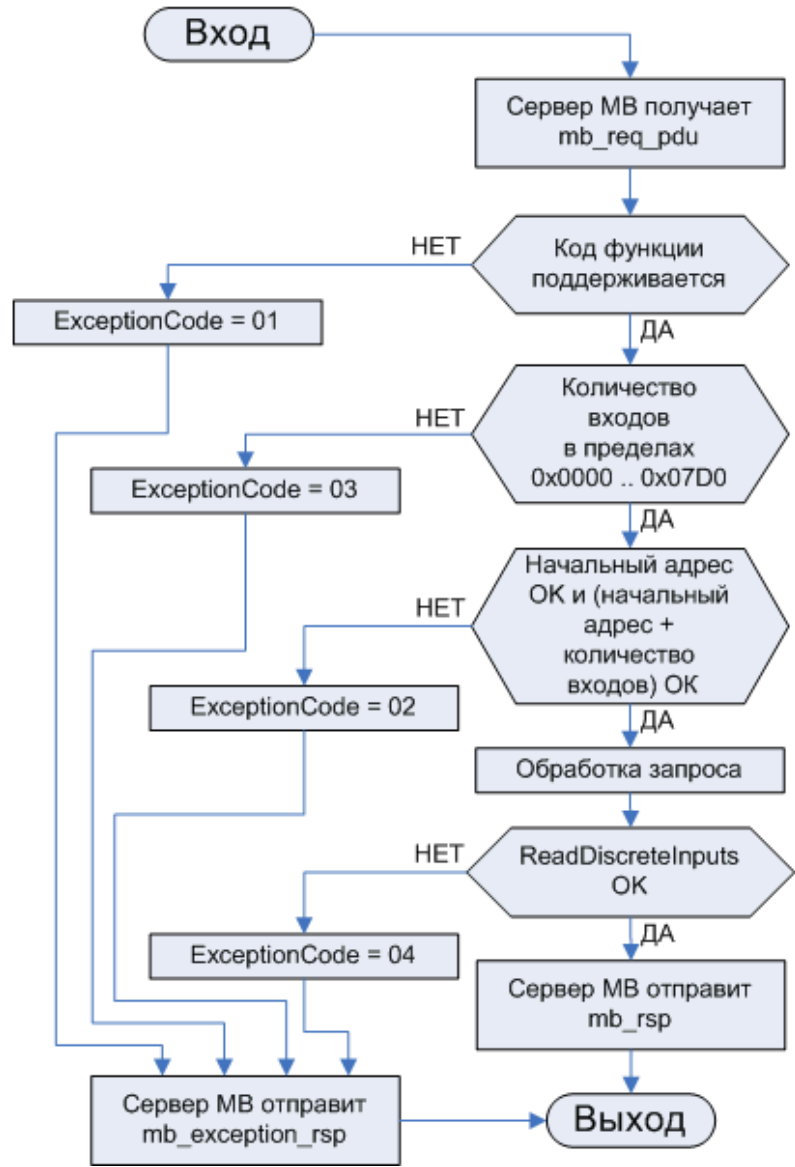


Рис. 12. Диаграмма состояний чтения входов (Read Discrete Inputs).

6.3. 03 (0x03) Read Holding Registers

Этот код функции используется для чтения содержимого блока смежных регистров временного хранения информации (holding registers) в сетевом устройстве. Запрос PDU задает начальный адрес регистра и количество регистров. В PDU регистры адресуются начиная с нуля. Таким образом, регистры с номерами 1 .. 16 адресуются как 0 .. 15.

Данные регистров в сообщении ответа упакованы как 2 байта на регистр, с двоичным содержанием, выравненным вправо в каждом байте. Для каждого регистра первый байт содержит старшие биты, и второй байт содержит младшие биты регистра.

Запрос:

Код функции	1 байт	0x03
Начальный адрес	2 байта	0x0000 .. 0xFFFF
Количество регистров	2 байта	1 .. 125 (0x0001 .. 0x007D)

Ответ:

Код функции	1 байт	0x03
Количество байт	1 байт	2 x N*
Значение регистров	N* x 2 байт	

Примечание *: N = количество регистров.

Ошибка:

Код функции	1 байт	0x83
Код исключения (exception code)	1 байт	Один из кодов от 01 до 04

Запрос	Ответ		
Имя поля	HEX	Имя поля	HEX
Функция	03	Функция	03
Старший байт адреса	00	Количество байт	06
Младший байт адреса	6B	Старший байт значения регистра 108	02
Старший байт количества регистров	00	Младший байт значения регистра 108	2B

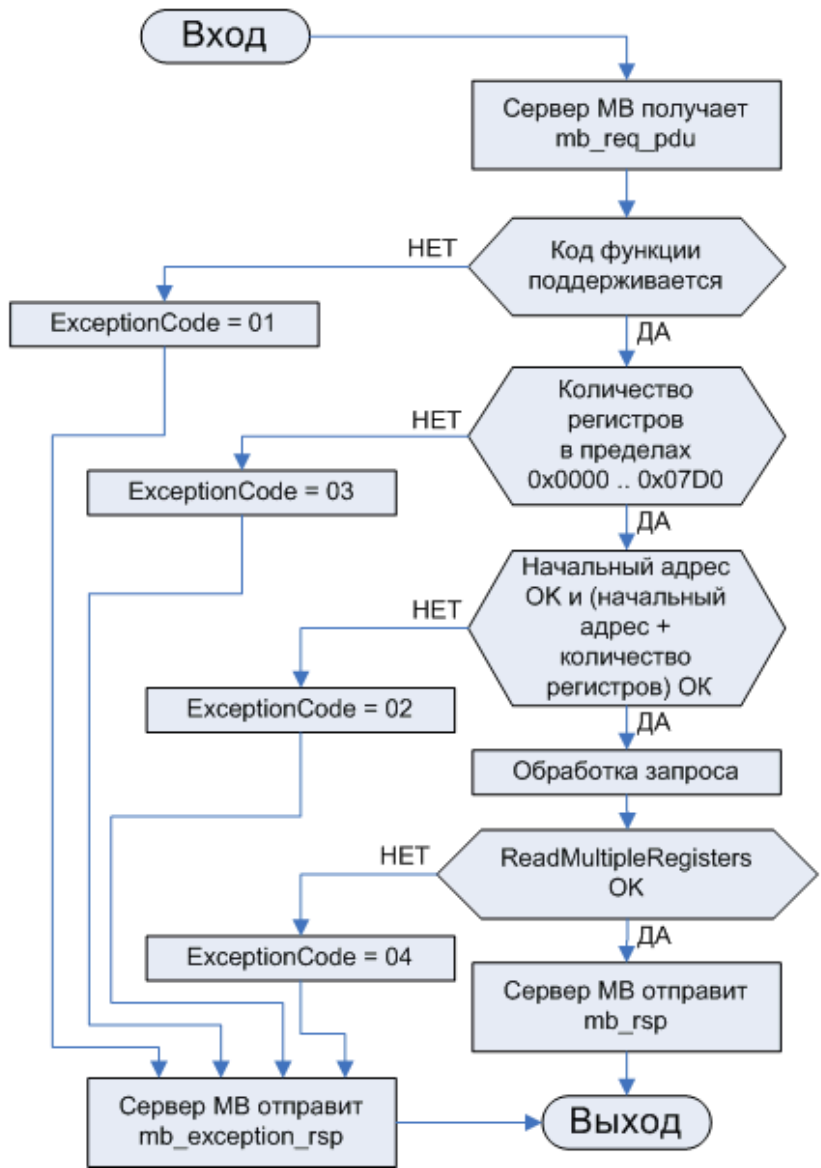


Рис. 13. Диаграмма состояний чтения регистров (Read Holding Registers).

6.4. 04 (0x04) Read Input Registers

Этот код функции используется для чтения 1 .. 125 смежных входных регистров сетевого устройства. Запрос PDU задает начальный адрес регистра и количество регистров. В PDU регистры адресуются начиная с нуля. Таким образом регистры, пронумерованные 1 .. 16 адресуются как 0 .. 15.

Данные регистров в сообщении ответа упакованы как 2 байта на регистр, с двоичным содержанием, выравненным вправо в каждом байте. Для каждого регистра первый байт содержит старшие биты, и второй байт содержит младшие биты регистра.

Запрос:

Код функции	1 байт	0x04
Начальный адрес	2 байта	0x0000 .. 0xFFFF
Количество входных регистров	2 байта	1 .. 125 (0x0001 .. 0x007D)

Ответ:

Код функции	1 байт	0x04
Количество байт	1 байт	2 x N*
Входные регистры	N* x 2 байт	

Примечание *: N = количество входных регистров.

Ошибка:

Код функции	1 байт	0x84
Код исключения (exсeption code)	1 байт	Один из кодов от 01 до 04

Пример запроса на чтение регистра 9

Запрос		Ответ	
Имя поля	HEX	Имя поля	HEX
Функция	04	Функция	04
Старший байт адреса	00	Количество байт	02
Младший байт адреса	08	Старший байт значения регистра 9	00
Старший байт количества регистров	00	Младший байт значения регистра 9	0A
Младший байт количества регистров	01		

Содержимое регистра 9 показано как 2 байта 0x00 0x0A (hex), или 10 в десятичном значении.

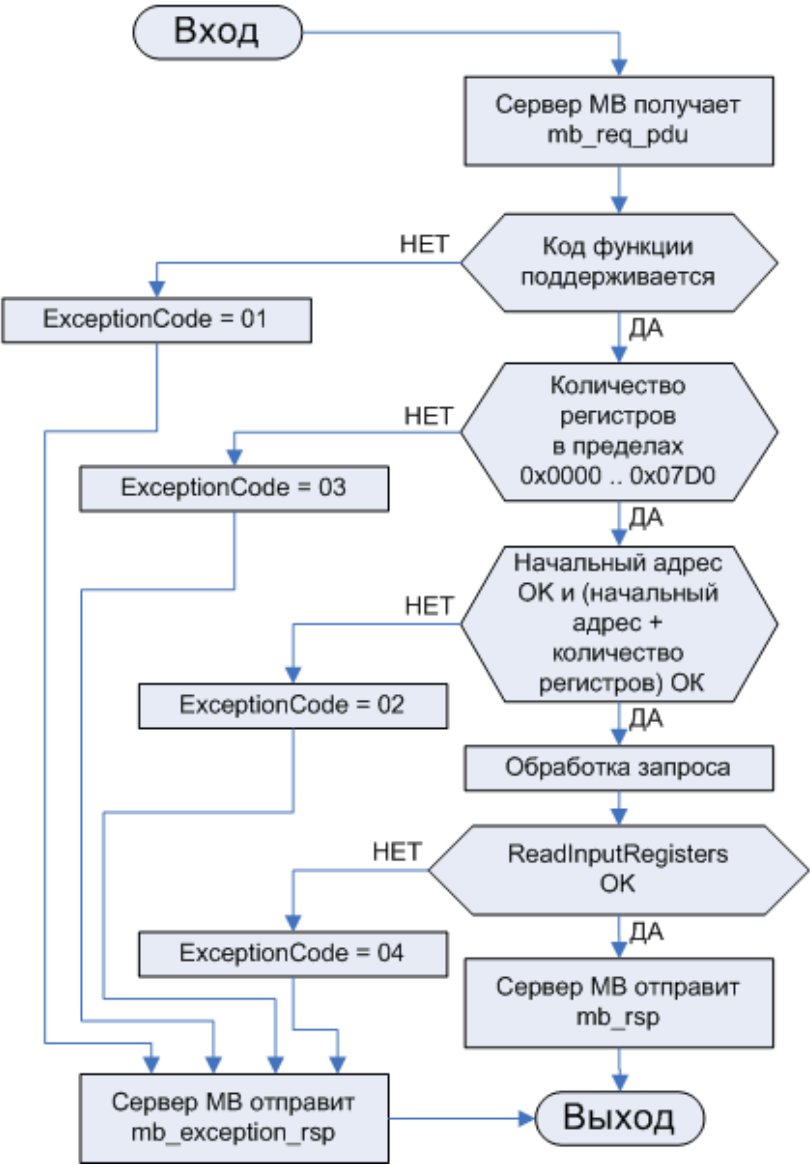


Рис. 14. Диаграмма состояний чтения входных регистров (Read Input Registers).

6.5. 05 (0x05) Write Single Coil

Этот код функции используется для записи одного выхода сетевого устройства в состояние ON или OFF. Запрошенное состояние ON/OFF задается константой в поле данных запроса. Значение запроса 0xFF 0x00 (hex) задает состояние ON выхода, а значение 0x00 0x00 задает OFF. Все другие значения недопустимы, и не влияют на выход.

Запрос PDU задает адрес обмотки, которая управляется. Обмотки адресуются начиная с нуля. Таким образом, обмотка с номером 1 адресуется как 0. Запрашиваемое состояние ON/OFF задается константой в поле Coil Value. Значение 0xFF00 соответствует ON, значение 0X0000 соответствует OFF. Все другие значения недопустимы, и не повлияют на состояние обмотки.

Нормальным ответом будет эхо запроса, возвращенное после того, как было записано состояние обмотки.

Запрос:

Код функции	1 байт	0x05
Адрес выхода	2 байта	0x0000 .. 0xFFFF
Значение выхода	2 байта	0 или 65280 (0x0000 или 0xFF00)

Ответ:

Код функции	1 байт	0x05
-------------	--------	-------------

Адрес выхода	2 байта	0x0000 .. 0xFFFF
Значение выхода	2 байта	0 или 65280 (0x0000 или 0xFF00)

Ошибка:

Код функции	1 байт	0x85
Код исключения (exception code)	1 байт	Один из кодов от 01 до 04

Пример запроса на запись Coil 173 ON

Запрос		Ответ	
Имя поля	HEX	Имя поля	HEX
Функция	05	Функция	05
Старший байт адреса	00	Старший байт адреса	00
Младший байт адреса	AC	Младший байт адреса	AC
Старший байт значения выхода	FF	Старший байт значения выхода	FF
Младший байт значения выхода	00	Младший байт значения выхода	00

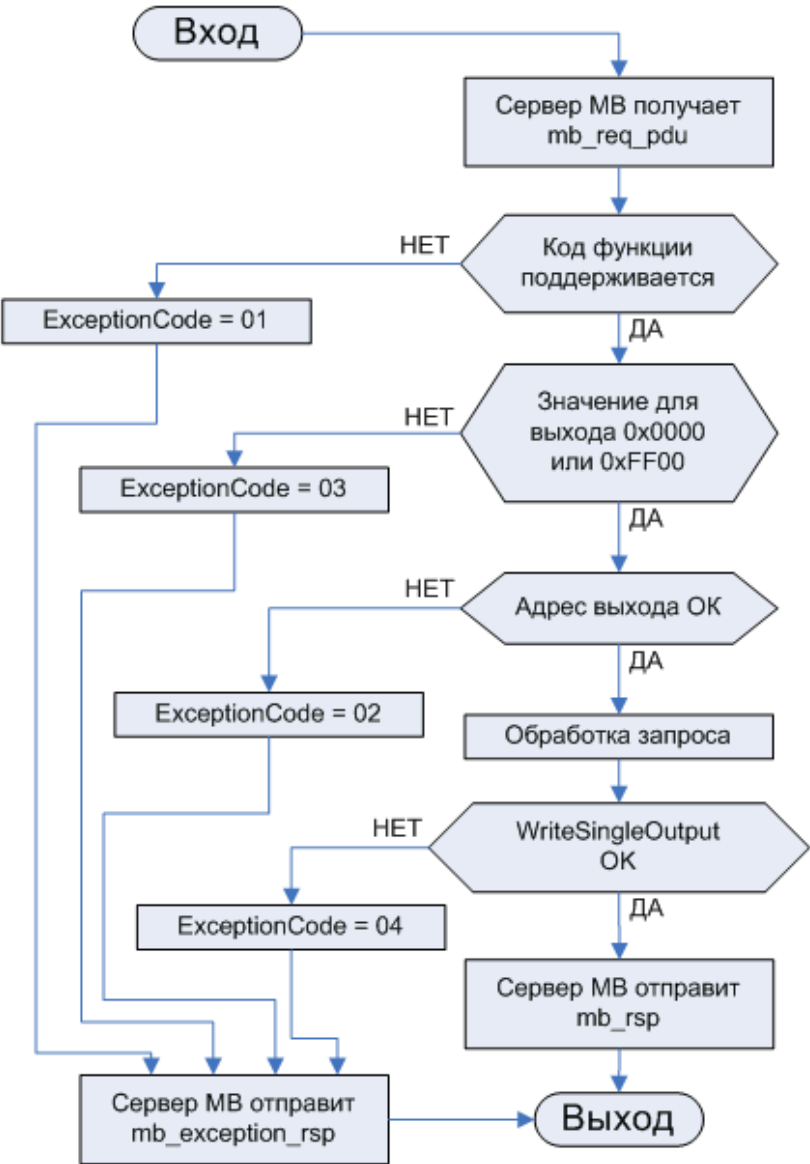


Рис. 15. Диаграмма состояний записи одного выхода.

6.6. 06 (0x06) Write Single Register

Этот код функции используется для записи одного регистра временного хранения в сетевом устройстве. Запрос PDU задает адрес записываемого регистра. Регистры адресуются начиная с 0. Таким образом регистр номер 1 адресуется как 0.

Нормальный ответ будет эхом запроса, возвращенным после того, как регистр был записан.

Запрос:

Код функции	1 байт	0x06
Адрес регистра	2 байта	0x0000 .. 0xFFFF
Значение регистра	2 байта	0x0000 .. 0xFFFF

Ответ:

Код функции	1 байт	0x06
Адрес регистра	2 байта	0x0000 .. 0xFFFF
Значение регистра	2 байта	0x0000 .. 0xFFFF

Ошибка:

Код функции	1 байт	0x86
Код исключения (exception code)	1 байт	Один из кодов от 01 до 04

Пример запроса на запись значения 0x0003 в регистр 2

Запрос		Ответ	
Имя поля	HEX	Имя поля	HEX
Функция	06	Функция	06
Старший байт адреса	00	Старший байт адреса	00
Младший байт адреса	01	Младший байт адреса	01
Старший байт значения регистра	00	Старший байт значения регистра	00
Младший байт значения регистра	03	Младший байт значения регистра	03

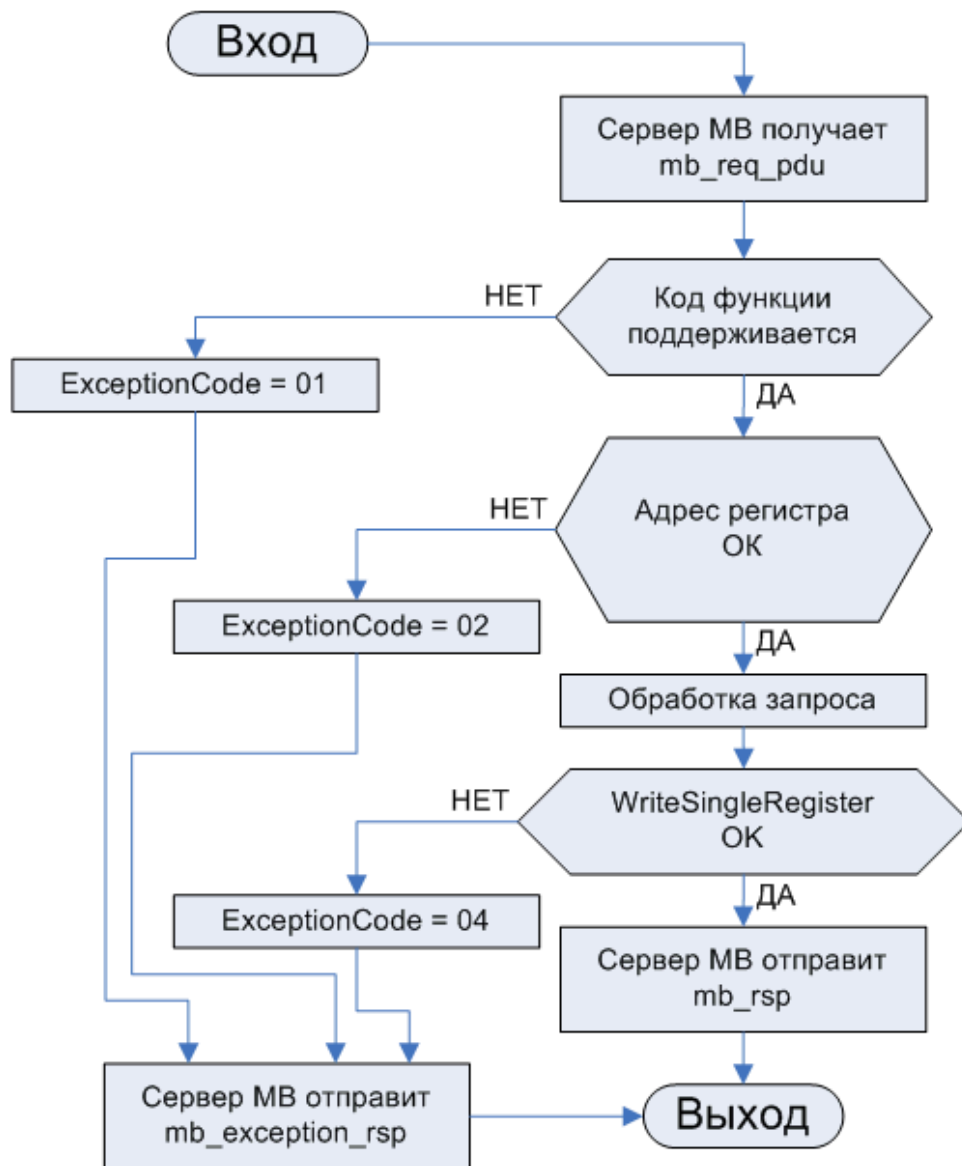


Рис. 16. Диаграмма состояний записи одного регистра.

6.7. 07 (0x07) Read Exception Status (только для последовательной линии)

Этот код функции используется для чтения восьми выходов состояния исключения (Exception Status) сетевого устройства.

Функция предоставляет простой метод доступа к этой информации, потому что ссылка на Exception Output известна (не нужно в функции задавать ссылку на выход).

Нормальный ответ содержит статус восьми выходов Exception Status. Выходы упакованы в один байт данных, по одному биту на выход. Статус младшего выхода соответствует LSB байта.

Содержимое восьми выходов Exception Status специфично для устройства.

Запрос:

Код функции	1 байт	0x07
-------------	--------	-------------

Ответ:

Код функции	1 байт	0x07
Выходные данные	1 байт	0x00 .. 0xFF

Ошибка:

Код функции	1 байт	0x87
Код исключения (exception code)	1 байт	01 или 04

Пример запроса на чтение Exception Status

Запрос		Ответ	
Имя поля	HEX	Имя поля	HEX
Функция	07	Функция	07
	00	Выходные данные	6D

В этом примере выходные данные 0x6D (hex), или в двоичном виде 01101101. Слева направо состояние выходов OFF–ON–ON–OFF–ON–ON–OFF–ON. Показано состояние от старшего до младшего адресованного выхода.

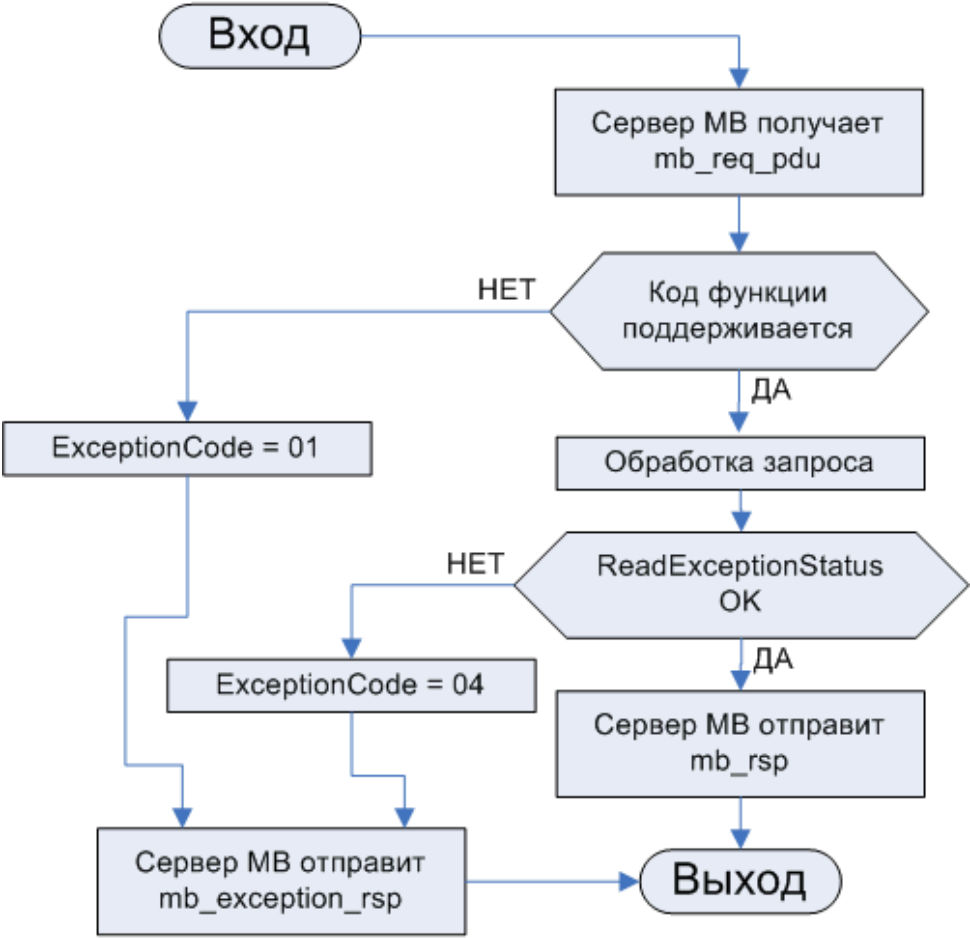


Рис. 17. Диаграмма состояний Read Exception Status.

6.8. 08 (0x08) Diagnostics (только для последовательной линии)

Код функции 08 предоставляет серии текстов для проверки системы коммуникации между устройством клиента (Master, главное устройство) и устройством сервера (Slave, подчиненное устройство), или для проверки различных внутренних состояний ошибки сервера.

Функция использует в запросе поле двухбайтного субкода функции, чтобы определить тип теста, который будет выполняться. Сервер в нормальном ответе эхом отправляет обратно клиенту код функции и код субфункции. Некоторые виды диагностики в поле данных нормального ответа возвращают информацию от сетевого устройства.

В общем выдача диагностической функции в сетевое устройство не влияет на его выполняемую рабочую программу. Тест не обращается к логике пользователя наподобие дискретных бит и регистров. Определенные функции могут опционально сбросить счетчики ошибок в сетевом устройстве.

Однако устройство сервера может быть переведено в режим "только прослушивание" (Listen Only Mode), в котором оно будет мониторить сообщения системы коммуникации, но не будет отвечать на них. Это может повлиять на результат работы прикладной программы, если это зависит от дальнейшего обмена данными с удаленным устройством. Как правило, этот режим принудительно удаляет неисправное удаленное устройство из системы связи (чтобы оно не мешало обмену данными с другими устройствами).

Для устройств последовательной линии определены следующие диагностические функции. Нормальный ответ на запрос Return Query Data возвращаются те же самые данные. Также эхом отправляются код функции и код субфункции.

Запрос:

Код функции	1 байт	0x08
Код субфункции	2 байта	
Данные	N x байт	

Ответ:

Код функции	1 байт	0x08
Код субфункции	2 байта	
Данные	N x байт	

Ошибка:

Код функции	1 байт	0x88
Код исключения (exception code)	1 байт	01, 03 или 04

6.8.1. Коды субфункций, поддерживаемые последовательными устройствами

В таблице ниже приведен список кодов субфункций, поддерживаемые устройствами последовательной линии. Затем каждый код субфункции приводится с примером содержимого поля данных, которое применимо для этой диагностики.

Код субфункции		Назначение
HEX	DEC	
00	00	Return Query Data (возврат запрошенных данных)
01	01	Restart Communications Option (перезапуск обмена данными)
02	02	Return Diagnostic Register (возврат регистра диагностики)
03	03	Change ASCII Input Delimiter (поменять ASCII-разделитель сообщений)
04	04	Force Listen Only Mode (перевести в неактивный режим коммуникаций, только прослушивание линии)
	05 .. 09	Зарезервировано
0A	10	Clear Counters and Diagnostic Register (очистка счетчиков и регистра диагностики)
0B	11	Return Bus Message Count (возврат счетчика сообщений на шине)
0C	12	Return Bus Communication Error Count (возврат счетчика ошибок обмена данными)
0D	13	Return Bus Exception Error Count (возврат счетчика исключений шины)
0E	14	Return Slave Message Count (возврат количества сообщений подчиненного устройства)
0F	15	Return Slave No Response Count (возврат счетчика запросов, на которые не было ответа)
10	16	Return Slave NAK Count (возврат количества отрицательных подтверждений)
11	17	Return Slave Busy Count (возврат количества сообщений занятости подчиненного устройства)
12	18	Return Bus Character Overrun Count (возврат количества символьных переполнений)
13	19	Зарезервировано

14	20	Clear Overrun Counter and Flag (очистка счетчика количества символьных переполнений и флага)
21 .. 65535		Зарезервировано

00 Return Query Data

Данные, переданные в поле данных запроса, возвращаются обратно в ответе. Все сообщение ответа должно быть идентично запросу.

Субфункция	Поле данных (запрос)	Поле данных (ответ)
00 00	Любые данные	Те же самые данные, что были в поле данных запроса

01 Restart Communications Option

Порт последовательной линии дальнего сетевого устройства должен быть инициализирован и перезапущен, и все коммуникационные счетчики событий очищены. Если порт в настоящее время находится в режиме "только прослушивание" (Listen Only Mode), то не этот субкод не последует ответа. Эта функция является единственной, которая выводит порт из режима "только прослушивание". Если порт находится не в режиме Listen Only, то последует нормальный ответ. Этот ответ будет отправлен перед рестартом последовательного порта.

Когда дальнее сетевое устройство приняло этот запрос, оно пытается перезапуститься и выполняет тесты проверки надежности рабочей среды при включении питания. Успешное завершение тестов переведет порт в рабочее состояние (online).

Если в поле данных запроса содержится 0xFF 0x00 (hex), то это приведет также к очистке лога событий обмена (Communications Event Log). Если в поле данных запроса 0x00 0x00, то содержимое лога останется неизменным, какое было до рестарта.

Субфункция	Поле данных (запрос)	Поле данных (ответ)
00 01	00 00	Те же самые данные, что были в поле данных запроса
00 01	FF 00	Те же самые данные, что были в поле данных запроса

02 Return Diagnostic Register

В ответе будет возвращено содержимое 16-битного регистра диагностики дальнего сетевого устройства.

Субфункция	Поле данных (запрос)	Поле данных (ответ)
00 02	00 00	Содержимое Diagnostic Register

03 Change ASCII Input Delimiter

Символ CHAR, переданный в поле данных запроса, становится маркером конца входного сообщения (разделителем сообщений) для всех будущих сообщений (заменяя этим символом символ LF по умолчанию). Эта функция полезна в случаях, когда LF (Line Feed, перевод строки 0x0A) не требуется в качестве маркера окончания сообщений ASCII.

Субфункция	Поле данных (запрос)	Поле данных (ответ)
00 03	CHAR 00	Те же самые данные, что были в поле данных запроса

04 Force Listen Only Mode

Принудительно переводит адресованное сетевое устройство в режим "только прослушивание" (Listen Only Mode) для коммуникаций MODBUS. Это изолирует устройство от других устройств в сети, позволяя им продолжать связь без прерывания связи из-за активности адресованного сетевого устройства. На этот субкод запроса не последует ответа.

Когда сетевое устройство переходит в режим Listen Only, выключаются все его активные блоки, управляющие обменом данными. Сторожевому таймеру готовности (Ready Watchdog) разрешено считать до таймаута, блокируя управление. Когда устройство находится в этом режиме, будут мониториться любые сообщения MODBUS, адресованные ему, а также широковещательные сообщения, однако никаких ответных действий предприниматься не будет, и ответ также посылаться не будет.

Единственная функция, которая будет обработана и выполнена после входа в этот режим - Restart Communications Option (код функции 8, код субфункции 1).

Субфункция	Поле данных (запрос)	Поле данных (ответ)
00 04	00 00	Нет никакого ответа

10 (0A Hex) Clear Counters and Diagnostic Register

Цель этой субфункции - очистить все счетчики и регистр диагностики. Счетчики также очищаются при включении питания.

Субфункция	Поле данных (запрос)	Поле данных (ответ)
00 0A	00 00	Те же самые данные, что были в поле данных запроса

11 (0B Hex) Return Bus Message Count

В поле данных ответа будет возвращено количество сообщений, которое было детектировано сетевым устройством с момента его последнего рестарта.

Субфункция	Поле данных (запрос)	Поле данных (ответ)
00 0B	00 00	Значение счетчика общего количества сообщений

12 (0C Hex) Return Bus Communication Error Count

В поле данных ответа будет возвращено количество ошибок CRC, произошедших на приеме сетевого устройства с момента его последнего рестарта, включения питания или операции по очистке счетчиков.

Субфункция	Поле данных (запрос)	Поле данных (ответ)
00 0C	00 00	Счетчик ошибок CRC

13 (0D Hex) Return Bus Exception Error Count

Поле данных ответа возвратит количество ответов исключений (MODBUS exception response), которые были возвращены с момента его последнего рестарта, включения питания или операции по очистке счетчиков. Ответы исключения описаны и перечислены в секции 7.

Субфункция	Поле данных (запрос)	Поле данных (ответ)
00 0D	00 00	Счетчик ошибок исключений

14 (0E Hex) Return Slave Message Count

Поле данные ответа возвратит количество сообщений, адресованных этому сетевому устройству, или широковещательных сообщений - т. е. количество всех сообщений, которое устройство обработало с момента его последнего рестарта, включения питания или операции по очистке счетчиков.

Субфункция	Поле данных (запрос)	Поле данных (ответ)
00 0E	00 00	Счетчик сообщений, обработанных подчиненным устройством (Slave Message Count)

15 (0F Hex) Return Slave No Response Count

В поле данных возвращается количество сообщений, адресованных этому сетевому устройству, на которые оно не вернуло ответ (ни нормальный ответ, ни ответ исключения) с момента его последнего рестарта, включения питания или операции по очистке счетчиков.

Субфункция	Поле данных (запрос)	Поле данных (ответ)
00 0F	00 00	Счетчик сообщений, на которые подчиненное устройство не выдало ответ (Slave No Response Count)

16 (10 Hex) Return Slave NAK Count

Поле данных ответа возвратит количество сообщений, адресованных в сетевому устройству, на которые оно отправило отрицательное подтверждение (Negative Acknowledge, NAK), ответ исключения, с момента его последнего рестарта, включения

питания или операции по очистке счетчиков. Ответы исключения описаны и перечислены в секции 7.

Субфункция	Поле данных (запрос)	Поле данных (ответ)
00 10	00 00	Счетчик отрицательных подтверждений

17 (11 Hex) Return Slave Busy Count

Поле данных ответа возвратит количество сообщений, адресованных этому сетевому устройству, на которые оно выдало ответ исключения занятости (Slave Device Busy) с момента его последнего рестарта, включения питания или операции по очистке счетчиков.

Субфункция	Поле данных (запрос)	Поле данных (ответ)
00 11	00 00	Счетчик сообщений о занятости подчиненного устройства

18 (12 Hex) Return Bus Character Overrun Count

Поле данных ответа возвратит количество сообщений, адресованных сетевому устройству, которые оно не обработало из-за события символьного переполнения (Character Overrun) с момента его последнего рестарта, включения питания или операции по очистке счетчиков. Символьное переполнение происходит, когда в порт слишком быстро поступают символы данных, или когда неправильно работает аппаратура.

Субфункция	Поле данных (запрос)	Поле данных (ответ)
00 12	00 00	Счетчик символьных переполнений

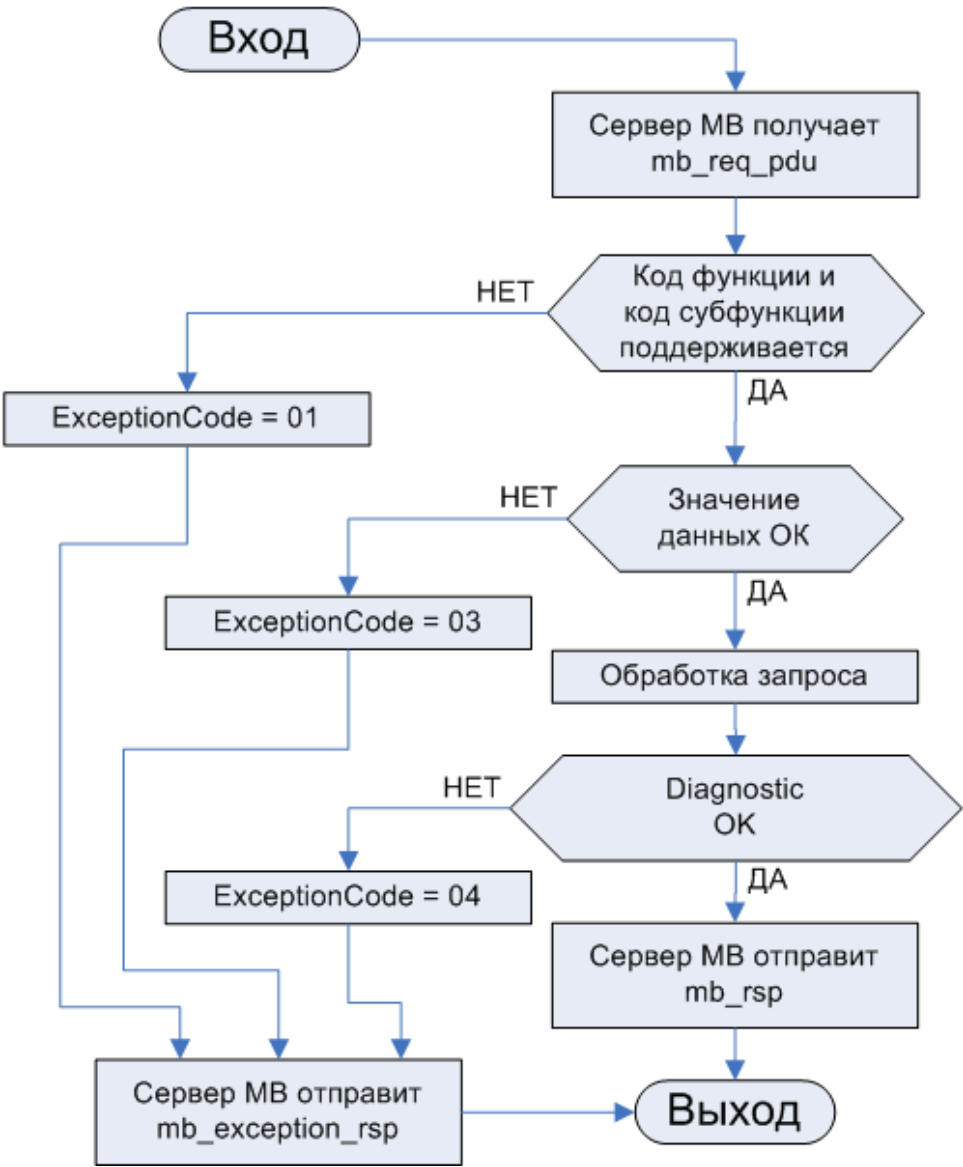


Рис. 18. Диаграмма состояний диагностики.

{/spoiler}

6.9. 11 (0x0B) Get Comm Event Counter (только для последовательной линии)

Этот код функции используется для получения слова статуса и счетчика событий коммуникаций (communication event counter) от сетевого устройства.

Путем получения текущего счетчика перед и после серии сообщений, клиент может определить, были ли сообщения нормально обработаны дальним сетевым устройством.

Счетчик событий устройства инкрементируется после каждого успешного завершения сообщения. Он не инкрементируется для ответов исключения, команд опроса или команд выдачи счетчика событий.

Счетчик событий может быть сброшен функцией диагностики (код функции 08), у которой код субфункции Restart Communications (код субфункции 00 01) или код субфункции Clear Counters and Diagnostic Register (код субфункции 00 0A).

Нормальный ответ содержит двухбайтное слово статуса и двухбайтный счетчик событий. Слово статуса будет всеми единицами FF FF (hex), если сетевым устройством все еще выполняется ранее выданная команда программы (состояние занятости). Иначе в слове статуса будут все нули.

Запрос:

Код функции	1 байт	0x0B
-------------	--------	------

Ответ:

Код функции	1 байт	0x0B
Статус	2 байта	0x0000 .. 0xFFFF
Счетчик событий	2 байта	0x0000 .. 0xFFFF

Ошибка:

Код функции	1 байт	0x8B
Код исключения (exception code)	1 байт	01 или 04

Пример запроса для получения счетчика событий от сетевого устройства

Запрос		Ответ	
Имя поля	HEX	Имя поля	HEX
Функция	0B	Функция	0B
		Старший байт статуса	FF
		Младший байт статуса	FF
		Старший байт счетчика событий	01
		Младший байт счетчика событий	08
В этом примере слово состояния 0xFF 0xFF (hex), что показывает выполняющуюся обработку в программе сетевого устройства. Счетчик событий показывает, что сетевым устройством было принято и подсчитано 264 (01 08 hex) события.			

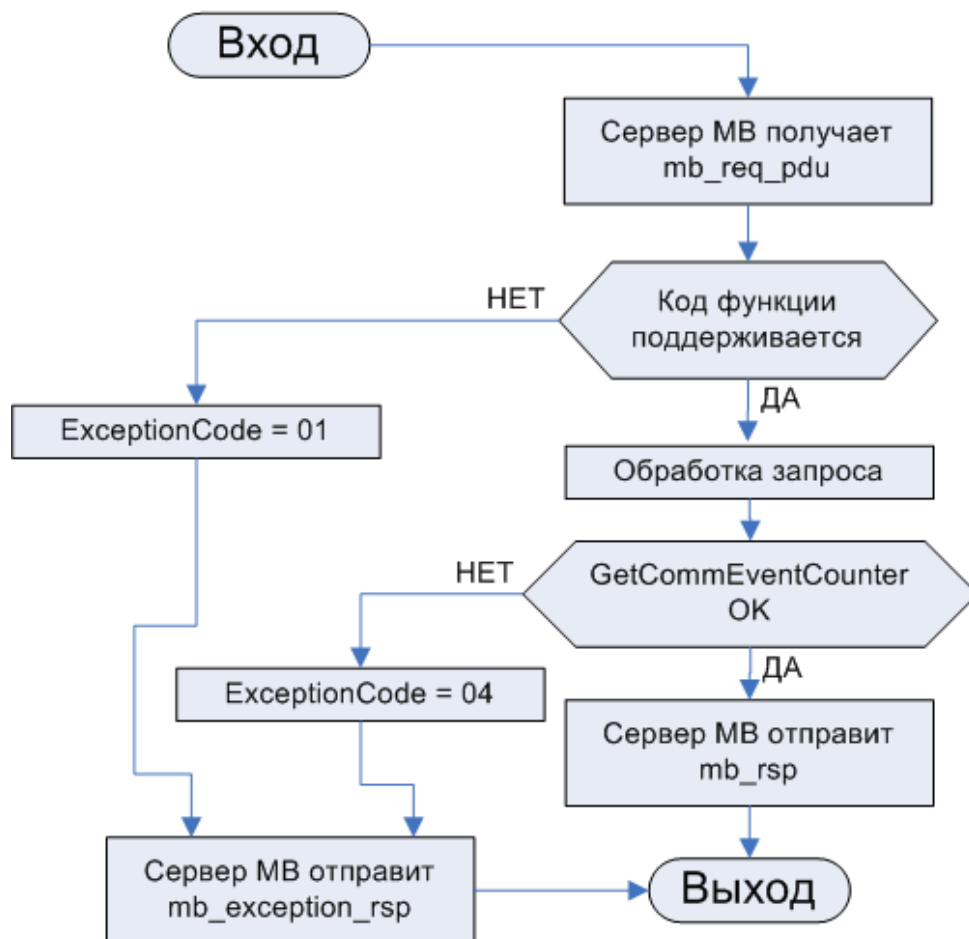


Рис. 19. Диаграмма состояний Get Comm Event Counter.

6.10. 12 (0x0C) Get Comm Event Log (только для последовательной линии)

Этот код функции используется для получения слова статуса, счетчика событий, счетчика сообщений и поля байт событий от сетевого устройства.

Слово статуса и счетчик событий идентичны тем, что возвращаются функцией Get Communications Event Counter function (11 0B hex).

Счетчик событий содержит количество сообщений, обработанных сетевым устройством с момента последнего рестарта. Этот счетчик идентичен тому, который возвращается функцией диагностики (код 08), субфункцией Return Bus Message Count (код 11, 0B hex).

Поле байт события содержит 0 .. 64 байт, где каждый байт соответствует статусу одной операции передачи или приема MODBUS сетевого устройства. Сетевое устройство вводит события в поле в хронологическом порядке. Байт 0 это самое свежее событие. Каждый новый байт выбрасывает самый старый байт из поля.

Нормальный ответ содержит двухбайтное поле слова статуса, двухбайтное поле счетчика событий и поле, содержащее 0 .. 64 байта событий. Поле количества байт содержит общую длину данных в этих четырех полях.

Запрос:

Код функции	1 байт	0x0C
-------------	--------	-------------

Ответ:

Код функции	1 байт	0x0C
Количество байт	1 байт	N*
Статус	2 байта	0x0000 .. 0xFFFF

Счетчик событий	2 байта	0x0000 .. 0xFFFF
Счетчик сообщений	2 байта	0x0000 .. 0xFFFF
События	(N-6) байт	

Примечание *: N = количество событий + 3 x 2 байт, (длина статуса, счетчик событий и счетчик сообщений).

Ошибка:

Код функции	1 байт	0x8C
Код исключения (exception code)	1 байт	01 или 04

Пример запроса на получение лога коммуникационных событий в сетевом устройстве

Запрос		Ответ	
Имя поля	HEX	Имя поля	HEX
Функция	0C	Функция	0C
		Количество байт	08
		Старший байт статуса	00
		Младший байт статуса	00
		Старший байт счетчика событий	01
		Младший байт счетчика событий	08
		Старший байт счетчика сообщений	01
		Младший байт счетчика сообщений	21
		Событие 0	20
		Событие 1	00

В этом примере слово статуса 00 00 (hex), которое показывает, что сетевое устройство не занято обработкой функции программы. Счетчик событий показывает 264 (01 08 hex) события, которые подсчитало сетевое устройство. Счетчик сообщений показывает, что 289 (01 21 hex) сообщений было обработано.

Самое свежее событие коммуникации показано в байте Event 0. Его содержимое (20 hex) показывает, что сетевое устройство только что вошло в режим Listen Only.

Предыдущее событие показано байтом Event 1. Его содержимое (00 hex) показывает, что сетевое устройство приняло Communications Restart.

Ниже дано описание содержимого байт события.

Что содержат байты события (Event Bytes). Байт события, возвращенный функцией Get Communications Event Log, может быть любым из четырех типов. Тип определяется битом 7 (старшим битом) в каждом байте. Тип может быть далее уточнен битом 6. Как это происходит, показано дальше.

• Remote device MODBUS Receive Event

Сетевое устройство сохраняет этот тип события, когда принято сообщение запроса. Сохранение происходит до того, как сетевое устройство обработает сообщение. Это событие определяется битом 7, установленным в лог. 1. Другие биты будут установлены в лог. 1, если соответствующее условие TRUE. Назначение разрядов бит:

Бит	Содержимое
0	Не используется
1	Ошибка обмена
2	Не используется
3	Не используется

- 4 Символьное переполнение (Character Overrun)
- 5 В настоящий момент действует режим Listen Only
- 6 Принято широковещательное сообщение (Broadcast)
- 7 1

• Remote device MODBUS Send Event

Сетевое устройство сохраняет этот тип события, когда оно завершило обработку сообщения запроса. Сохранение происходит, если устройство возвратило нормальный ответ или ответ исключения, или не ответило. Это событие определено битом 7, установленным в лог. 0 и с битом 6, установленным в лог. 1. Другие биты будут установлены в лог. 1, когда соответствующее условие TRUE. Назначение разрядов бит:

Бит **Содержимое**

- 0 Read Exception Sent (отправлено исключение чтения, коды исключений 1 .. 3)
- 1 Slave Abort Exception Sent (отправлено исключение обрыва связи подчиненного устройства, код исключения 4)
- 2 Slave Busy Exception Sent (отправлено исключение занятости, коды исключений 5 и 6)
- 3 Slave Program NAK Exception Sent (отправлено исключение отрицательного подтверждения, код исключения 7)
- 4 Write Timeout Error Occurred (произошла ошибка таймаута)
- 5 В настоящий момент действует режим Listen Only
- 6 1
- 7 0

• Remote device Entered Listen Only Mode

Сетевое устройство сохраняет этот тип события, когда оно входит в режим Listen Only. Это событие определено содержимым 0x04 (hex).

• Remote device Initiated Communication Restart

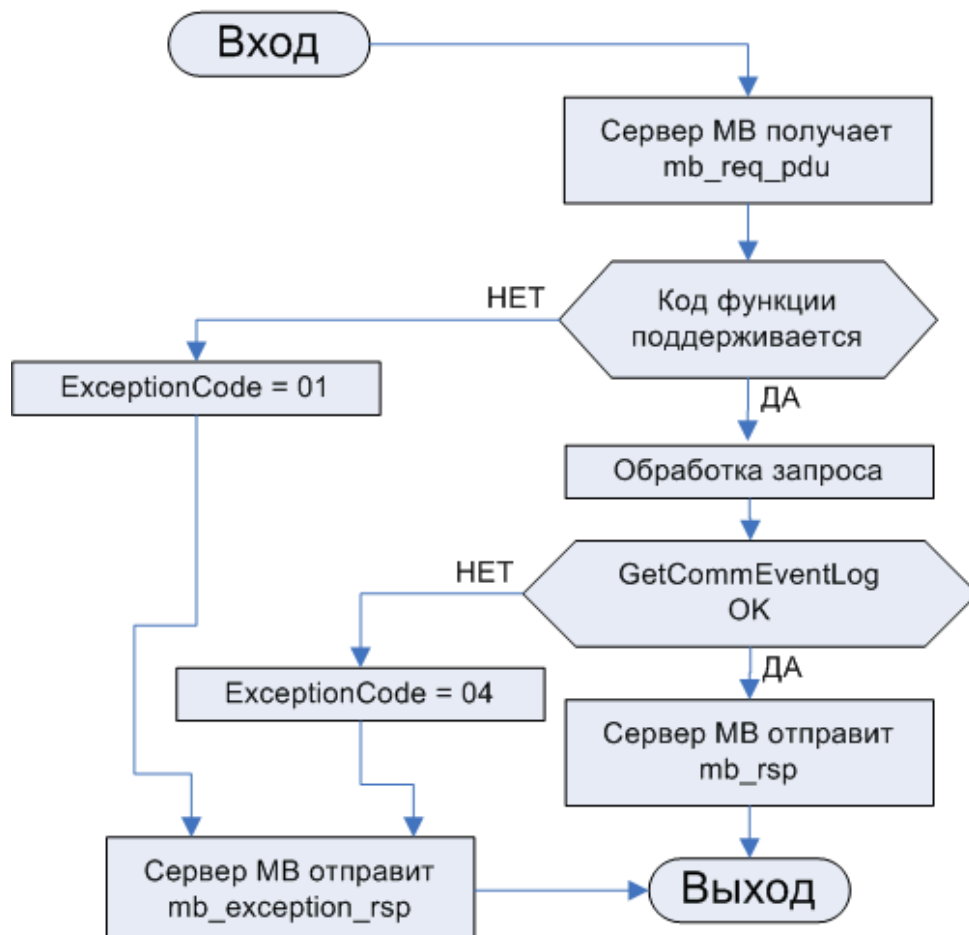


Рис. 20. Диаграмма состояний Get Comm Event Log.

6.11. 15 (0x0F) Write Multiple Coils

Этот код функции используется в сетевом устройстве для переключения его каждой обмотки в последовательности либо в состояние ON, либо в состояние OFF. Запрос PDU задает обмотки, которые управляются. Обмотки адресуются начиная с 0. Таким образом, обмотка номер 1 адресуется как 0.

Запрашиваемые состояния ON/OFF задаются содержимым поля данных. Лог. 1 в позиции бита запрашивает перевод соответствующего выхода в состояние ON. Лог. 0 запрашивает перевод выхода в состояние OFF.

Нормальный ответ возвратит код функции, начальный адрес и количество управляемых обмоток.

Запрос:

Код функции	1 байт	0x0F
Начальный адрес	2 байта	0x0000 .. 0xFFFF
Количество выходов	2 байта	1 .. 1968 (0x0001 .. 0x07B0)
Количество байт	1 байт	N*
Значение выхода	N байт	

Примечание *: N = Количество выходов / 8, если остаток отличается от 0, то N = N+1.

Ответ:

Код функции	1 байт	0x0C
Начальный адрес	2 байта	0x0000 .. 0xFFFF
Количество выходов	2 байта	1 .. 1968 (0x0001 .. 0x07B0)

Ошибка:

Код функции	1 байт	0x8F
Код исключения (exception code)	1 байт	01 или 02 или 03 или 04

Пример запроса записи последовательности 10 обмоток, начиная с обмотки 20

Содержимое данных в запросе представлено двумя байтами 0xCD 0x01 (hex), в двоичном виде 1100 1101 0000 0001. Биты соответствуют выходам следующим образом (черточки означают, что эти биты не используются для управления выходами):

Бит:	1	1	0	0	1	1	0	1	0	0	0	0	0	0	1
Выход:	27	26	25	24	23	22	21	20	-	-	-	-	-	29	28

Первый передаваемый байт (0xCD hex) адресует выходы 27 .. 20, где LSB адресует самый младший управляемый выход (20).

Следующий передаваемый байт (01 hex) адресует выходы 29 и 28, где LSB адресует выход (28) в этой последовательности. Не используемые биты (их выходы обозначены черточками) должны быть обнулены.

Запрос		Ответ	
Имя поля	HEX	Имя поля	HEX
Функция	0F	Функция	0F
Старший байт начального адреса	00	Старший байт начального адреса	00
Младший байт начального адреса	13	Младший байт начального адреса	13

Старший байт количества выходов	00	Старший байт количества выходов	00
Младший байт количества выходов	0A	Младший байт количества выходов	0A
Количество байт	02		
Старший байт значения выходов	CD		

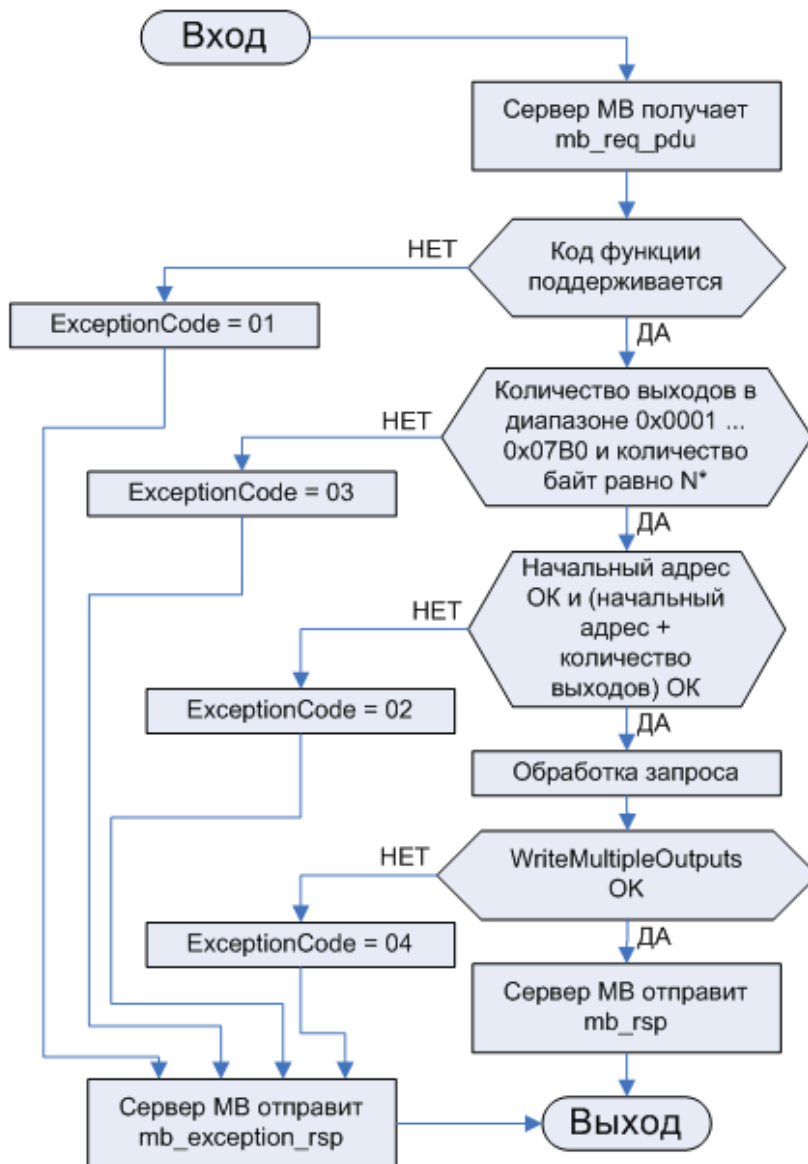


Рис. 21. Диаграмма состояний Write Multiple Outputs.

6.12. 16 (0x10) Write Multiple registers

Этот код функции используется для записи блока смежных регистров (1 .. 123 регистров) в сетевом устройстве.

Запрашиваемые записываемые значения задаются полем данных запроса. Данные упакованы как 2 байта на регистр.

Нормальный ответ вернет код функции, начальный адрес и количество записанных регистров.

Запрос:

Код функции	1 байт	0x10
Начальный адрес	2 байта	0x0000 .. 0xFFFF
Количество регистров	2 байта	1 .. 123 (0x0001 .. 0x007B)
Количество байт	1 байт	2 x N*
Значения регистров	N x 2 байт	

Примечание *: N = количество регистров.

Ответ:

Код функции	1 байт	0x10
Начальный адрес	2 байта	0x0000 .. 0xFFFF
Количество регистров	2 байта	1 .. 123 (0x0001 .. 0x007B)

Ошибка:

Код функции	1 байт	0x90
Код исключения (exception code)	1 байт	01 или 02 или 03 или 04

Пример запроса записи двух регистров, начиная с адреса 2, в значения 00 0A и 01 02 (hex)

Запрос		Ответ	
Имя поля	HEX	Имя поля	HEX
Функция	10	Функция	10
Старший байт начального адреса	00	Старший байт начального адреса	00
Младший байт начального адреса	01	Младший байт начального адреса	01
Старший байт количества регистров	00	Старший байт количества регистров	00
Младший байт количества регистров	02	Младший байт количества регистров	02
Количество байт	04		
Старший байт значения регистра	00		
Младший байт значения регистра	0A		
Старший байт значения регистра	01		
Младший байт значения регистра	02		

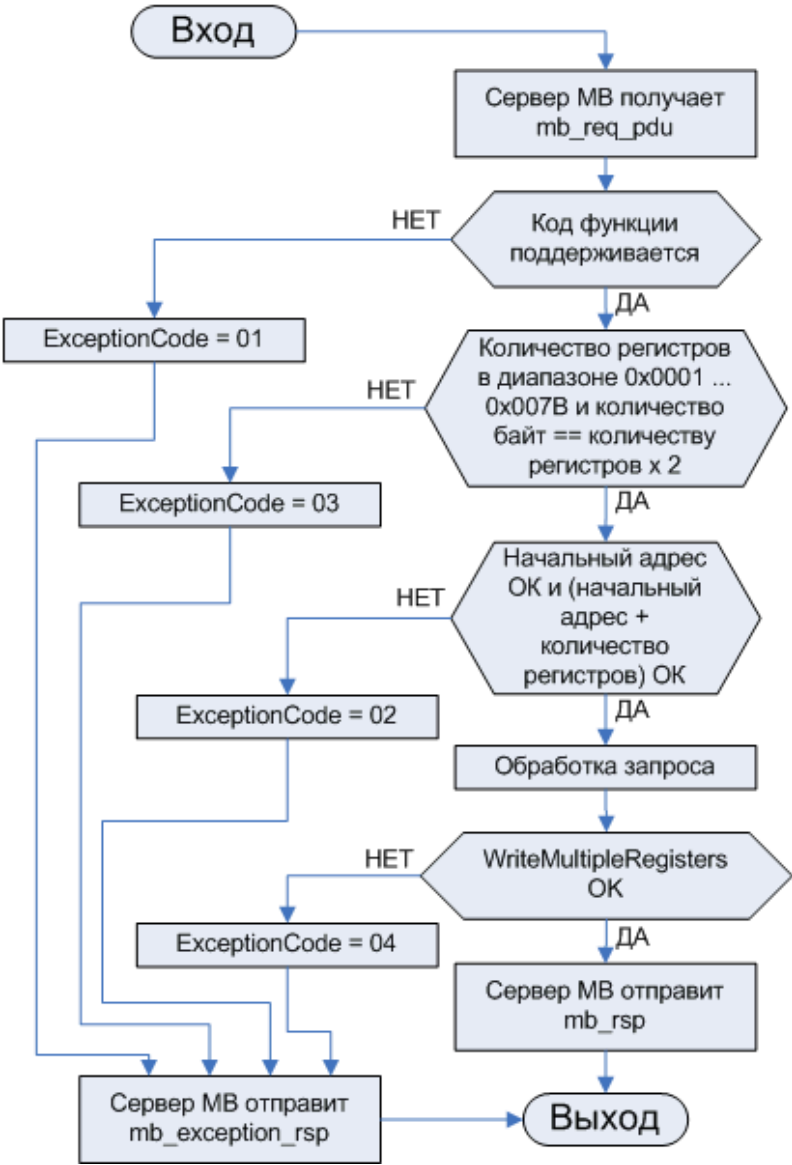


Рис. 22. Диаграмма состояний Write Multiple Registers.

6.13. 17 (0x11) Report Slave ID (только для последовательной линии)

Этот код функции используется для чтения описания типа, текущего состояния и других информационных параметров, специфичных для сетевого устройства.

Запрос:

Код функции	1 байт	0x11
-------------	--------	-------------

Ответ:

Код функции	1 байт	0x10
Количество байт	1 байт	Зависит от устройства
Slave ID		Зависит от устройства
Run Indicator Status (индикатор активности устройства)	1 байт	0x00 OFF, 0xFF ON
Дополнительные данные		Зависит от устройства

Ошибка:

Код функции	1 байт	0x91
-------------	--------	-------------

Пример Report Slave ID

Формат нормального ответа показан в следующем примере (ID и статус). Содержимое данных ответа специфично для каждого типа устройства.

Запрос		Ответ	
Имя поля	HEX	Имя поля	HEX
Функция	11	Функция	11
		Количество байт	Зависит от устройства
		Slave ID	Зависит от устройства
		Run Indicator Status	0x00 или 0xFF
		Дополнительные данные	Зависит от устройства

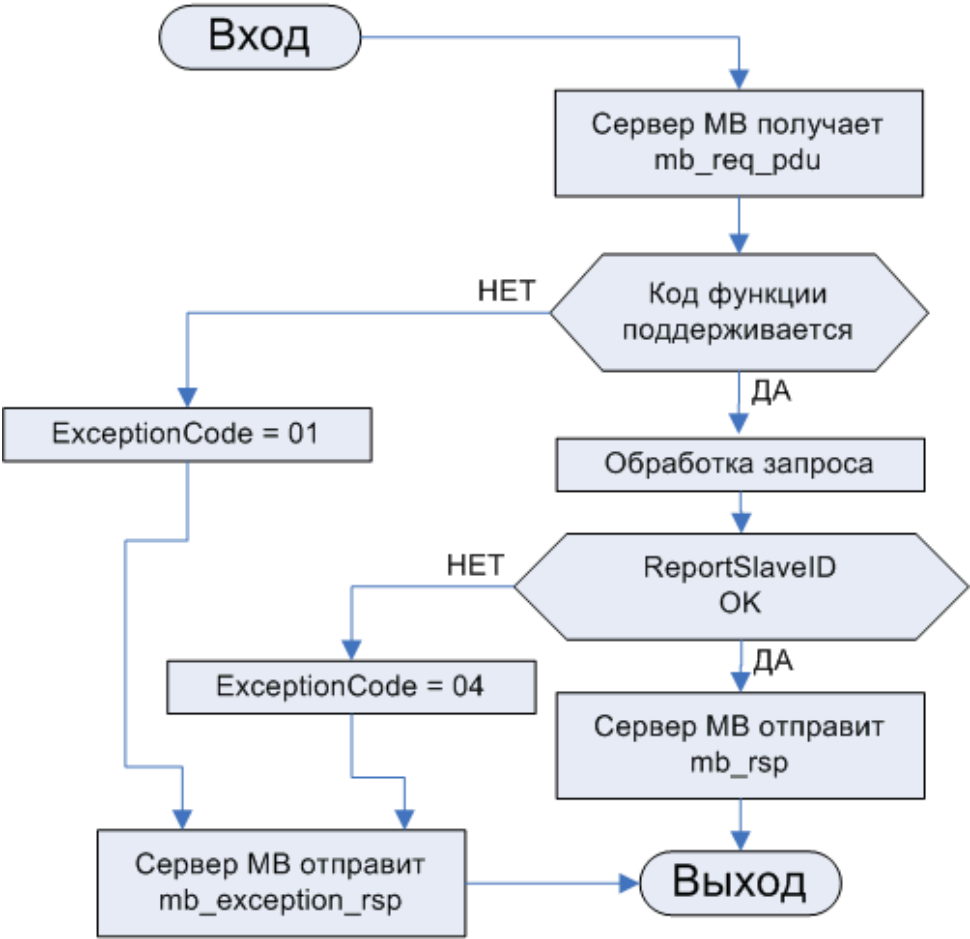


Рис. 23. Диаграмма состояний Report slave ID.

6.14. 20 (0x14) Read File Record

Этот код функции используется для чтения из файла. Все запросы длин данных (Request Data Lengths) предоставляются в единицах байт, и все запросы длин записей (Record Lengths) предоставляются в единицах регистров.

Файл это организация записей. Каждый файл содержит 10000 записей, адресованных десятичными значениями от 0000 до 9999 или HEX-значениями от 0x0000 до 0x270F. Например, запись 12 адресуется как 12.

Функция может прочитать несколько групп ссылок. Группы могут быть разделены (быть не смежными), однако элементы в каждой группе должны быть последовательными.

Каждая группа определяется отдельным полем субзапроса, которое содержит 7 байт:

Тип запроса: 1 байт (должен быть указан как 6)

Номер файла: 2 байта

Начальный номер записи в файле: 2 байта

Длина записи для чтения: 2 байта.

Количество регистров для чтения, в комбинации со всеми другими полями в ожидаемом ответе, не должно превышать допустимой длины MODBUS PDU: 253 байта.

Нормальный ответ это серия субответов, по одному на каждый субзапрос. Поле количества байт это общее скомбинированное количество байт во всех субответах. Дополнительно каждый субответ содержит поле, которое показывает свое количество байт.

Запрос:

Код функции	1 байт	0x14
Количество байт	1 байт	7 .. 245 (0x07 .. 0xF5)
Sub-Req. x, Reference Type	1 байт	06
Sub-Req. x, File Number	2 байта	0x0001 .. 0xFFFF
Sub-Req. x, Record Number	2 байта	0 .. 9999 (0x0000 .. 0x270F)
Sub-Req. x, Record Length	2 байта	N
Sub-Req. x+1, ...		

Ответ:

Код функции	1 байт	0x14
Длина данных ответа	1 байт	7 .. 245 (0x07 .. 0xF5)
Sub-Req. x, длина файла в ответе	1 байт	7 .. 245 (0x07 .. 0xF5)
Sub-Req. x, Reference Type	1 байт	6
Sub-Req. x, Record Data	N x 2 байт	
Sub-Req. x+1, ...		

Ошибка:

Код функции	1 байт	0x94
Код исключения (exception code)	1 байт	01 или 02 или 03 или 04 или 08

Примечание: хотя допускается задавать номер файла в диапазоне 1 .. 0xFFFF, следует заметить, что совместимость с устаревшим оборудованием может быть нарушена, если номер файла превышает 10 (0x0A).

Пример запроса на чтение двух групп ссылок из сетевого устройства

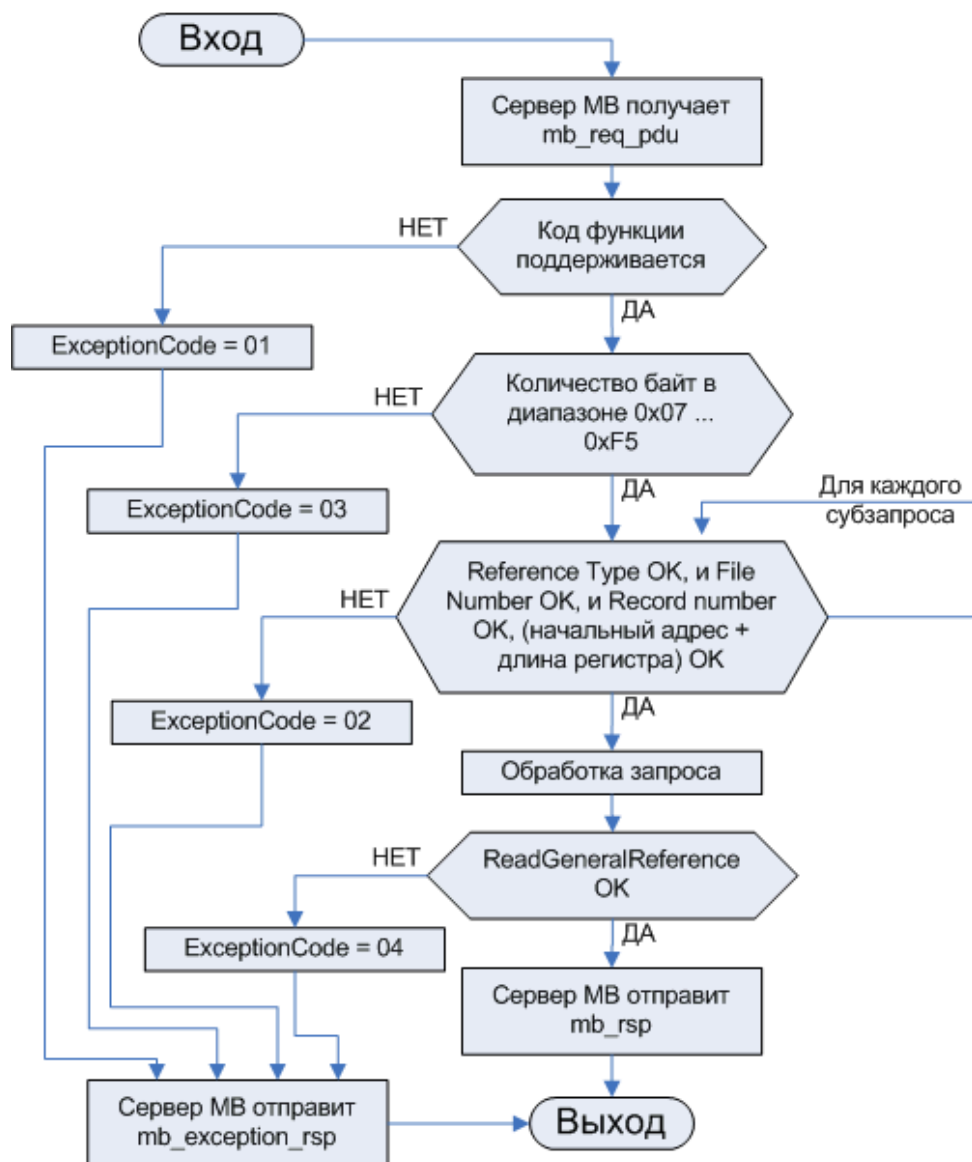


Рис. 24. Диаграмма состояний Read File Record.

6.15. 21 (0x15) Write File Record

Этот код функции используется для выполнения записи в файл. Все длины данных запроса (Request Data Lengths) предоставляются в байтах, и все длины записей (Record Lengths) в единицах 16-битных слов.

Файл это организация записей. Каждый файл содержит 10000 записей, адресованных десятичными значениями от 0000 до 9999 или HEX-значениями от 0X0000 до 0X270F. Например, запись 12 адресуется как 12.

Функция может записать несколько групп ссылок. Группы могут быть разделены (быть не смежными), однако элементы в каждой группе должны быть последовательными.

Каждая группа определена в отдельном поле субзапроса, в котором 7 байт плюс данные:

Тип запроса: 1 байт (должен быть указан как 6).

Номер файла: 2 байта.

Начальный номер записи в файле: 2 байта.

Длина для записи: 2 байта.

Записываемые данные: по 2 байта на регистр.

Количество регистров для записи, в комбинации со всеми другими полями в ожидаемом ответе, не должно превышать допустимой длины MODBUS PDU: 253 байта.

Нормальный ответ будет эхом (копией) запроса.

Запрос:

Код функции	1 байт	0x15
Длина данных запроса	1 байт	9 .. 251 (0x09 .. 0xFB)
Sub-Req. x, Reference Type	1 байт	06
Sub-Req. x, File Number	2 байта	0x0001 .. 0xFFFF
Sub-Req. x, Record Number	2 байта	0 .. 9999 (0x0000 .. 0x270F)
Sub-Req. x, Record Length	2 байта	N
Sub-Req. x, Record data	N x 2 байт	
Sub-Req. x+1, ...		

Ответ:

Код функции	1 байт	0x15
Длина данных запроса	1 байт	9 .. 251 (0x09 .. 0xFB)
Sub-Req. x, Reference Type	1 байт	06
Sub-Req. x, File Number	2 байта	0x0001 .. 0xFFFF
Sub-Req. x, Record Number	2 байта	0 .. 9999 (0x0000 .. 0x270F)
Sub-Req. x, Record Length	2 байта	N
Sub-Req. x, Record data	N x 2 байт	
Sub-Req. x+1, ...		

Ошибка:

Код функции	1 байт	0x95
Код исключения (exception code)	1 байт	01 или 02 или 03 или 04 или 08

Примечание: хотя допускается задавать номер файла в диапазоне 1 .. 0xFFFF, следует заметить, что что совместимость с устаревшим оборудованием может быть нарушена, если номер файла превышает 10 (0x0A).

Пример запроса записи группы элементов в сетевое устройство

Группа состоит из трех регистров в файле 4, начиная с регистра 7 (адрес 0007).

Запрос		Ответ	
Имя поля	HEX	Имя поля	HEX
Функция	15	Функция	15
Длина данных запроса	0D	Длина данных запроса	0D
Sub-Req. 1, Ref. Type	06	Sub-Req. 1, Ref. Type	06
Sub-Req. 1, старший байт номера файла	00	Sub-Req. 1, старший байт номера файла	00
Sub-Req. 1, младший байт номера файла	04	Sub-Req. 1, младший байт номера файла	04
Sub-Req. 1, старший байт номера элемента файла	00	Sub-Req. 1, старший байт номера элемента файла	00
Sub-Req. 1, младший байт номера элемента файла	07	Sub-Req. 1, младший байт номера элемента файла	07
Sub-Req. 1, старший байт длины элемента файла	00	Sub-Req. 1, старший байт длины элемента файла	00
Sub-Req. 1, младший байт длины элемента файла	03	Sub-Req. 1, младший байт длины элемента файла	03
Sub-Req. 1, старший байт данных регистра	06	Sub-Req. 1, старший байт данных регистра	06
Sub-Req. 1, младший байт данных регистра	AF	Sub-Req. 1, младший байт данных регистра	AF

Sub-Req. 1, старший байт данных регистра	04	Sub-Req. 1, старший байт данных регистра	04
Sub-Req. 1, младший байт данных регистра	BE	Sub-Req. 1, младший байт данных регистра	BE
Sub-Req. 1, старший байт данных регистра	10	Sub-Req. 1, старший байт данных регистра	10
Sub-Req. 1, младший байт данных регистра	0D	Sub-Req. 1, младший байт данных регистра	0D

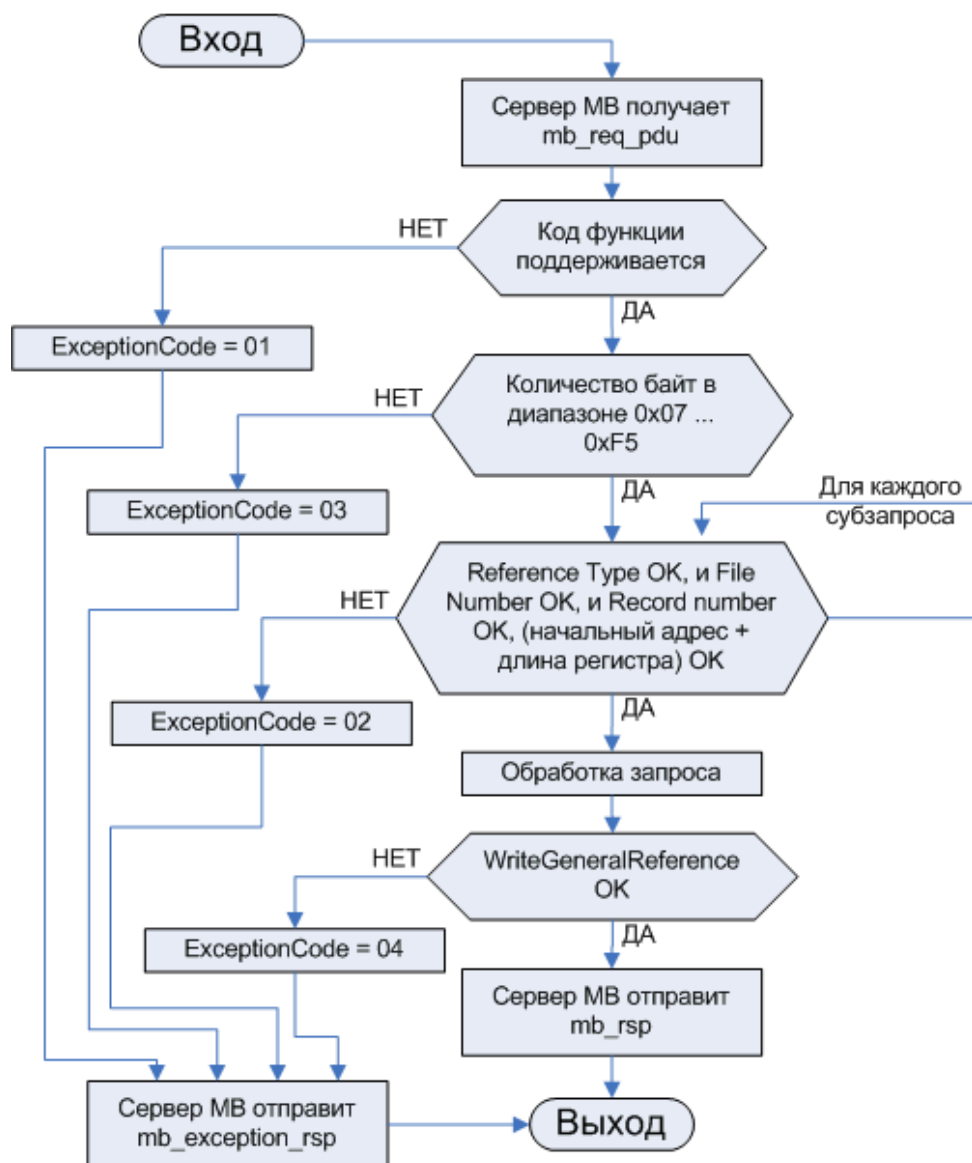


Рис. 25. Диаграмма состояний Write File Record.

6.16. 22 (0x16) Mask Write Register

Этот код функции используется для модификации содержимого регистра временного хранения (holding register), используя комбинацию маски И (AND), маски ИЛИ (OR) и текущего содержимого регистра. Эта функция может использоваться для установки или очистки отдельных бит в регистре.

Запрос задает временный регистр хранения, который будет записываться, данные для маски AND и данные для маски OR. Регистры адресуются начиная с 0. Таким образом, регистры 1 .. 16 адресуются как 0 .. 15.

Алгоритм функции:

Результат = (текущее содержимое & маскаAND) | (маскаOR & (^маскаAND))

Пример:

Компоненты операции	HEX	BIN
Текущее содержимое регистра	12	00010010
маскаAND	F2	11110010
маскаOR	25	00100101
(^маскаAND)	0D	00001101
Результат	17	00010111

Примечание: если значение маскаOR равно 0, то результатом будет простое логическое И текущего содержимого регистра и значения маскаAND. Если значение маскаAND равно 0, то результат будет равен значению маскаOR. Содержимое регистра может быть прочитано функцией Read Holding Registers (код функции 03). Однако регистр (регистры) могут быть изменены впоследствии в результате работы логики программы пользователя.

Нормальный ответ будет эхом (копией) запроса.

Запрос:

Код функции	1 байт	0x16
Адрес регистра	2 байта	0x0000 .. 0xFFFF
маскаAND	2 байта	0x0000 .. 0xFFFF
маскаOR	2 байта	0x0000 .. 0xFFFF

Ответ:

Код функции	1 байт	0x16
Адрес регистра	2 байта	0x0000 .. 0xFFFF
маскаAND	2 байта	0x0000 .. 0xFFFF
маскаOR	2 байта	0x0000 .. 0xFFFF

Ошибка:

Код функции	1 байт	0x96
Код исключения (exception code)	1 байт	01 или 02 или 03 или 04

Пример Mask Write

Запись осуществляется в регистр 5 сетевого устройства, используя маски из примера выше:

Запрос		Ответ	
Имя поля	HEX	Имя поля	HEX
Функция	16	Функция	16
Старший байт адреса	00	Старший байт адреса	00
Младший байт адреса	04	Младший байт адреса	04
Старший байт маскаAND	00	Старший байт маскаAND	00
Младший байт маскаAND	F2	Младший байт маскаAND	F2
Старший байт маскаOR	00	Старший байт маскаOR	00
Младший байт маскаOR	25	Младший байт маскаOR	25

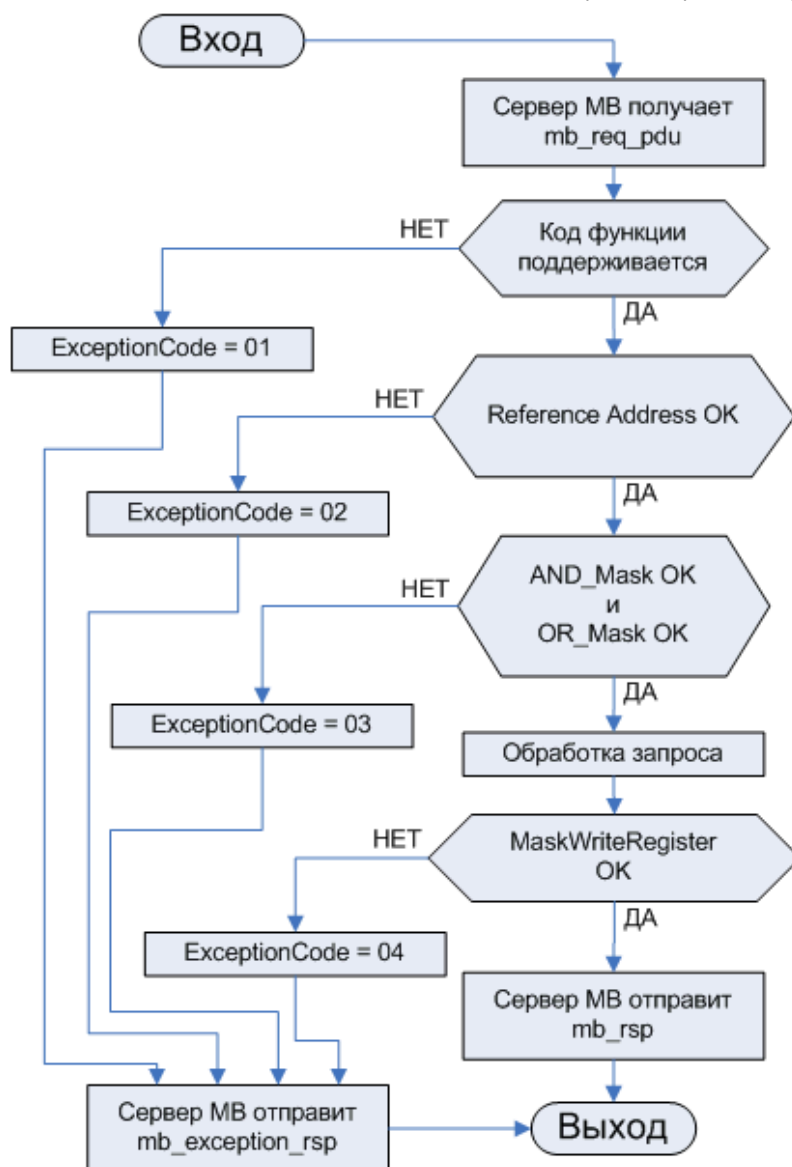


Рис. 26. Диаграмма состояний Mask Write Holding Register.

6.17. 23 (0x17) Read/Write Multiple registers

Этот код функции выполняет комбинацию одной операции чтения и одной операции записи в одной транзакции MODBUS. Операция записи выполняется перед операцией чтения.

Регистры временного хранения (holding registers) адресуются начиная с 0. Таким образом, регистры 1 .. 16 адресуются в PDU как 0 .. 15.

Запрос задает как начальный адрес и количество holding-регистров для чтения, так и начальный адрес и количество holding-регистров для записи. Количество байт задает, сколько байт следует в поле записи данных.

Нормальный ответ содержит данные из группы регистров, которая была прочитана. Поле количества байт задает, сколько байт следует в поле данных чтения.

Запрос:

Код функции	1 байт	0x17
Начальный адрес чтения	2 байта	0x0000 .. 0xFFFF
Количество читаемых регистров	2 байта	1 .. 125 (0x0001 .. 0x007D)
Начальный адрес записи	2 байта	0x0000 .. 0xFFFF
Количество записываемых регистров	2 байта	1 .. 121 (0x0001 .. 0x0079)

Количество записываемых байт	1 байт	2 x N*
Значения записываемых регистров	N x 2 байт	

Примечание *: N = количество регистров для записи.

Ответ:

Код функции	1 байт	0x17
Количество байт	1 байт	2 x N*
Значения прочитанных регистров	N x 2 байт	

Примечание *: N = количество регистров для чтения.

Ошибка:

Код функции	1 байт	0x97
Код исключения (exception code)	1 байт	01 или 02 или 03 или 04

Пример запроса на чтение и запись регистров

6 регистров считываются начиная с регистра 4, и записываются 3 регистра, начиная с регистра 15:

Запрос		Ответ	
Имя поля	HEX	Имя поля	HEX
Функция	17	Функция	17
Старший байт начального адреса чтения	00	Количество байт	0C
Младший байт начального адреса чтения	03	Старший байт считанного регистра	00
Старший байт количества читаемых регистров	00	Младший байт считанного регистра	FE
Младший байт количества читаемых регистров	06	Старший байт считанного регистра	0A
Старший байт начального адреса записи	00	Младший байт считанного регистра	CD
Младший байт начального адреса записи	0E	Старший байт считанного регистра	00
Старший байт количества записываемых регистров	00	Младший байт считанного регистра	01
Младший байт количества записываемых регистров	03	Старший байт считанного регистра	00
Количество записываемых байт	06	Младший байт считанного регистра	03
Старший байт записываемого регистра	00	Старший байт считанного регистра	00
Младший байт записываемого регистра	FF	Младший байт считанного регистра	0D
Старший байт записываемого регистра	00	Старший байт считанного регистра	00
Младший байт записываемого регистра	FF	Младший байт считанного регистра	FF
Старший байт записываемого регистра	00		
Младший байт записываемого регистра	FF		

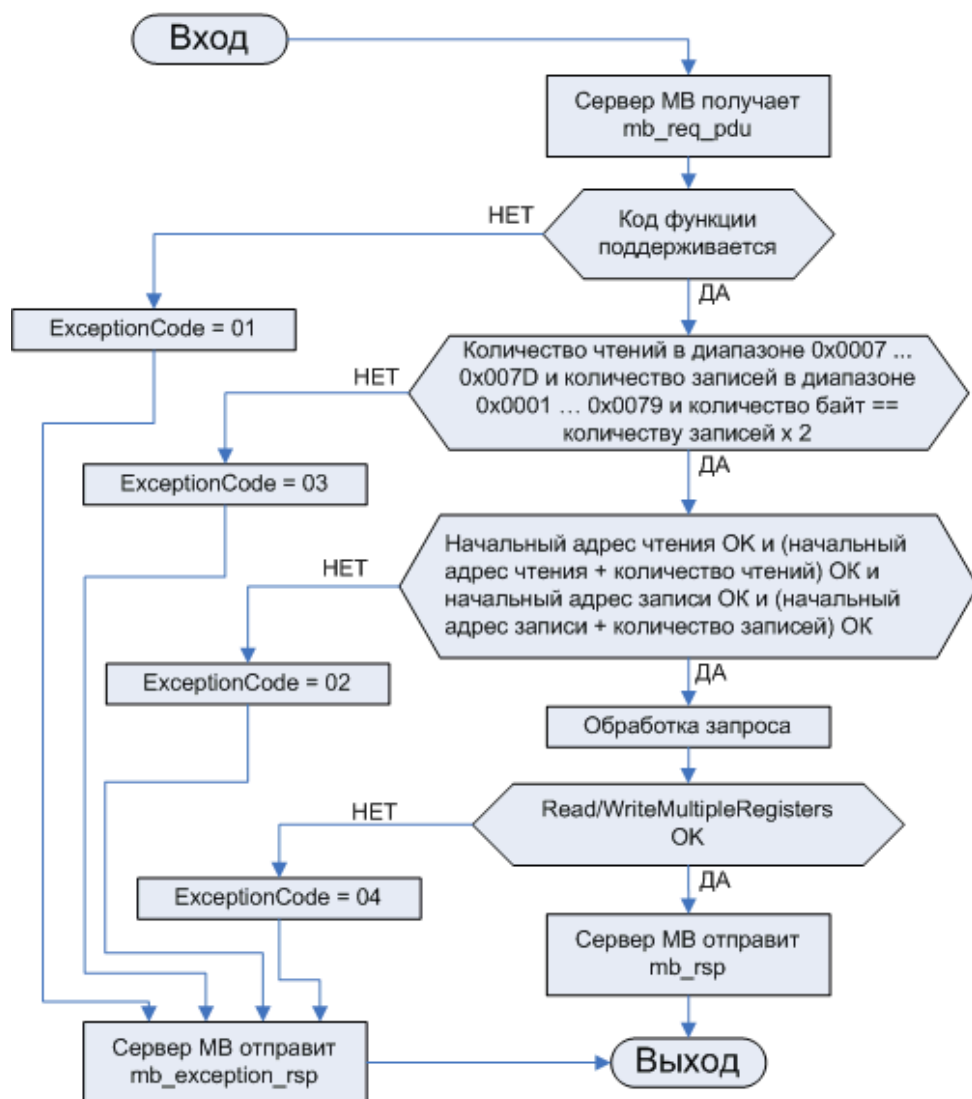


Рис. 27. Диаграмма состояния Read/Write Multiple Registers.

6.18. 24 (0x18) Read FIFO Queue

Этот код функции позволяет прочитать содержимое очереди FIFO регистров сетевого устройства. Функция вернет количество регистров в очереди, за которым будут идти данные очереди. Можно прочитать до 32 регистров: количество плюс 31 данных поставленных в очередь регистров. Сначала возвращается количество регистров в очереди, за ним следуют данные регистров в очереди.

Функция считывает содержимое очереди, однако не очищает её.

В нормальном ответе количество байт (Byte Count) показывает, сколько байт будет идти дальше, включая количество байт очереди и значение байт регистра (но не включая поля проверки на ошибку).

FIFO Count это количество регистров данных в очереди (не включая регистра количества).

Если FIFO Count превышает 31, то будет возвращен ответ исключения с кодом ошибки 03 (Illegal Data Value).

Запрос:

Код функции	1 байт	0x18
Адрес указателя FIFO	2 байта	0x0000 .. 0xFFFF

Ответ:

Код функции	1 байт	0x18
-------------	--------	-------------

Количество байт	2 байта	
Количество регистров FIFO	2 байта	≤ 31
Значения регистров FIFO	N* x 2 байт	

Пример *: N = количеству регистров в FIFO.

Ошибка:

Код функции	1 байт	0x98
Код исключения (exception code)	1 байт	01 или 02 или 03 или 04

Пример запроса Read FIFO Queue

Запрос выполнит чтение очереди, начиная с регистра указателя 1246 (0x04DE hex):

Запрос		Ответ	
Имя поля	HEX	Имя поля	HEX
Функция	18	Функция	18
Старший байт указателя адреса FIFO	04	Старший байт количества байт	00
Младший байт указателя адреса FIFO	DE	Младший байт количества байт	06
		Старший байт количества регистров FIFO	00
		Младший байт количества регистров FIFO	02
		Старший байт значения регистра FIFO	01
		Младший байт значения регистра FIFO	B8
		Старший байт значения регистра FIFO	12
		Младший байт значения регистра FIFO	84

В этом примере регистр указателя FIFO (1246) будет возвращен с количеством в очереди 2. За количеством идут два регистра данных очереди:

1247 (содержимое 440 десятичное, или 0x01B8 hex)
1248 (содержимое 4740 десятичное, или 0x1284 hex)

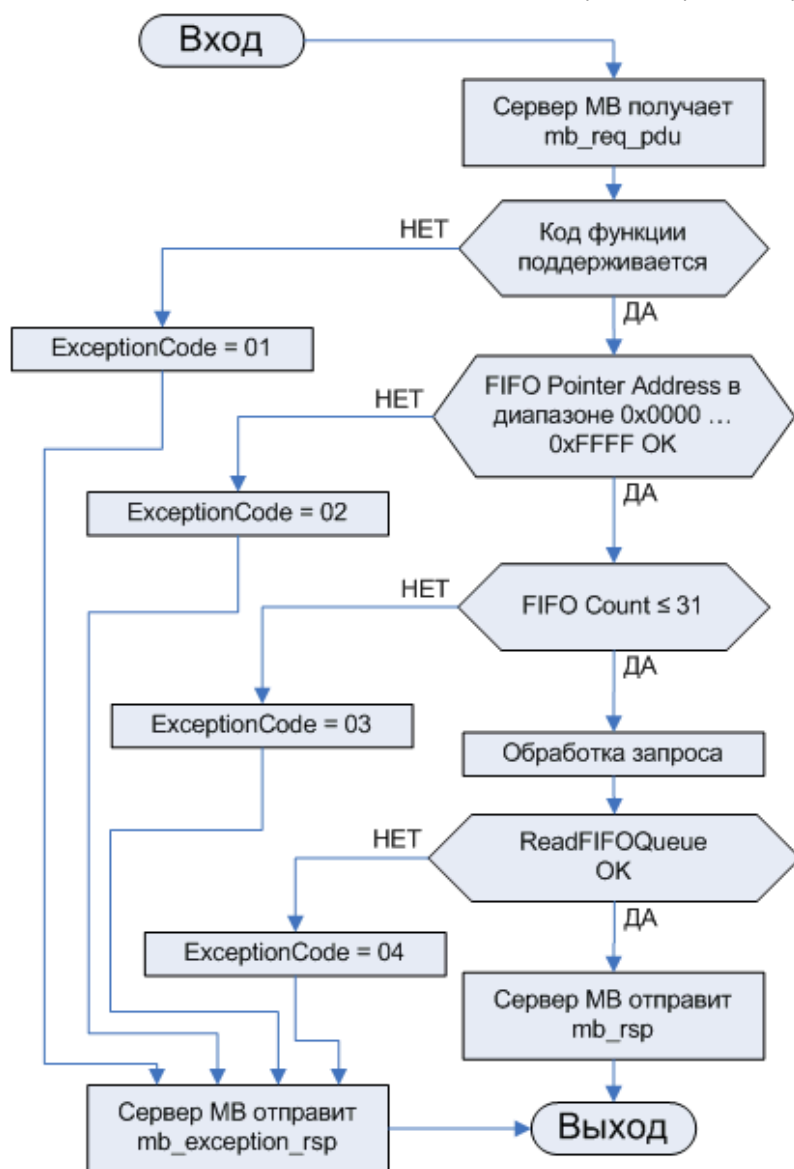


Рис. 28: Диаграмма состояний Read FIFO Queue.

6.19. 43 (0x2B) Encapsulated Interface Transport

Примечание: в Приложении А находится информация по зарезервированным кодам MODBUS, субкодам и типам MEI.

Код функции 43 и его тип MEI 14 для идентификации устройства это один из двух инкапсулированных транспорта интерфейса (Encapsulated Interface Transport), доступных сейчас в этой спецификации [1]. Следующие коды функций и типы MEI не входят как часть этой публичной спецификации и эти коды функций и типы MEI зарегистрированы специально: 43 / 0 .. 12 и 43 / 15 .. 255.

Транспорт MODBUS Encapsulated Interface (MEI) это механизм туннелирования запросов службы и запуска методов, а также их возвращаемых данных, внутри MODBUS PDU.

Главное назначение транспорта MEI - инкапсуляция вызовов методов или запросов служб, которые являются частью определенного интерфейса, а также возвращаемых результатов метода или ответов службы.

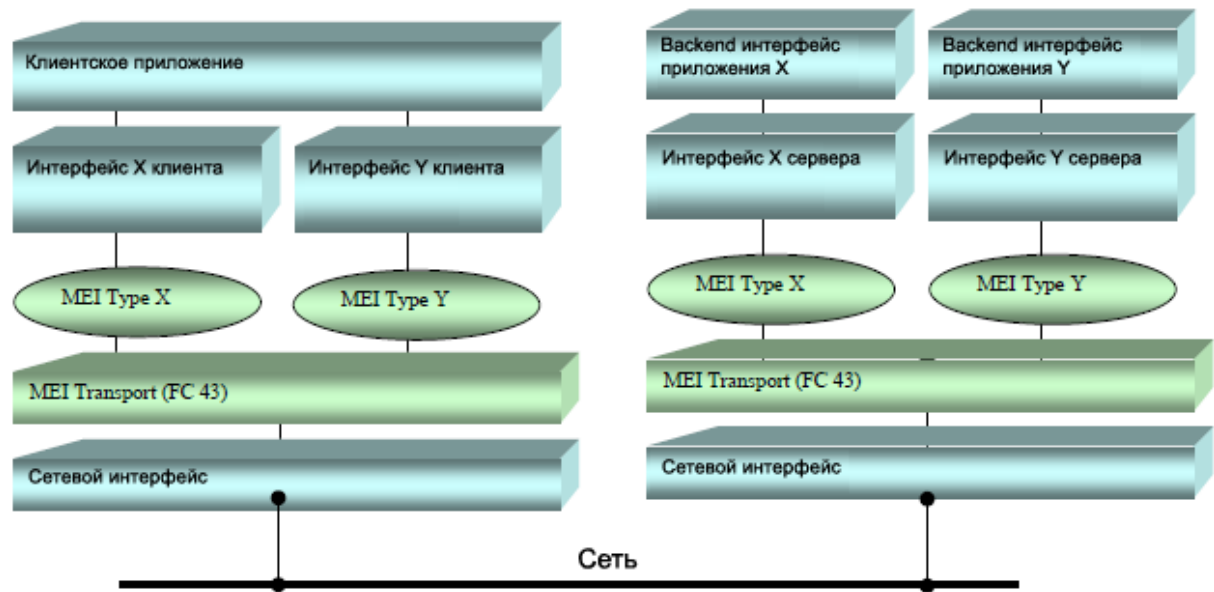


Рис. 29: Транспорт MEI (MODBUS encapsulated Interface).

Сетевой интерфейс может быть любым коммуникационным стеком, использующимся для отправки MODBUS PDU, как например TCP/IP или последовательная линия связи.

MEI Type (тип MEI) это специально назначенный номер (MODBUS Assigned Number) и поэтому он будет уникальным, значение между 0 и 255 зарезервировано согласно Приложению А, кроме MEI Type 13 и MEI Type 14.

MEI Type используется реализациями транспорта MEI для диспетчеризации вызова метода на показанном интерфейсе.

Поскольку служба транспорта MEI отвязана от интерфейса, любое специфическое поведение или политика, требуемая интерфейсом, должна предоставляться самим интерфейсом, т. е. обработка транзакции MEI, обработка ошибок интерфейса MEI, и т. п.

Запрос:

Код функции	1 байт	0x2B
MEI Type	1 байт	0x0D или 0x0E
Специфические данные типа MEI	n байт	

Ответ:

Код функции	1 байт	0x2B
MEI Type	1 байт	То же самое значение, что было в запросе
Специфические данные типа MEI	n байт	

Ошибка:

Код функции	1 байт	0xAB (результат 0x2B + 0x80)
Код исключения (exception code)	1 байт	01 или 02 или 03 или 04

В качестве примера см. запрос Read Device Identification.

6.20. 43 / 13 (0x2B / 0x0D) CANopen General Reference Request and Response PDU

CANopen General reference Command это инкапсуляция служб, которые будут использоваться для доступа (на чтение или запись) элементов словаря объектов CANopen (CANopen Device Object Dictionary), а также для управления и мониторинга системы CANopen и её устройств.

MEI Type 13 (0x0D) это MODBUS Assigned Number, лицензированный CiA для CANopen General Reference.

Система предназначена для работы в пределах ограничений существующих сетей MODBUS. Поэтому информация, необходимая для запроса или изменения объектных словарей CANopen в системе, преобразуется в формат сообщения MODBUS. PDU будет ограничен 253 байтами как в сообщении запроса, так и в сообщении ответа.

На веб-сайте MODBUS-IDA или на веб-сайте CiA (CAN in Automation) можно ознакомиться с копиями и условиями использования, охватывающими код функции 43 MEI Type 13.

6.21 43 / 14 (0x2B / 0x0E) Read Device Identification

Этот код функции позволяет прочитать идентификацию и дополнительную информацию, относящуюся только к физическому и функциональному описанию сетевого устройства.

Интерфейс Read Device Identification смоделирован как адресное пространство, составленное из набора адресуемых элементов данных. Элементы данных называются объектами, и их идентифицирует object Id.

Интерфейс состоит из 3 категорий объектов:

- Basic Device Identification (базовая идентификация устройства). Все объекты в этой категории обязательны: имя производителя (VendorName), код изделия (Product code) и номер ревизии (revision number).
- Regular Device Identification (обычная идентификация устройства). В дополнение к базовым объектам данных, устройство предоставляет дополнительную и опциональную идентификацию и описание объектов данных. Все объекты в этой категории определены в стандарте, но их реализация не является обязательной.
- Extended Device Identification (расширенная идентификация устройства). В дополнение к обычным объектам данных, устройство предоставляет дополнительную и опциональную идентификацию и описание приватных данных о самом физическом устройстве. Все эти данные зависят от конкретного устройства.

Object Id	Имя объекта / описание	Тип	Наличие	Категория
0x00	VendorName (имя производителя)	Строка ASCII	Обязательно	Basic
0x01	ProductCode (код продукта)			
0x02	MajorMinorRevision (ревизия)			
0x03	VendorUrl (ссылка на сайт производителя)		Не обязательно	Regular
0x04	ProductName (имя продукта)			
0x05	ModelName (имя модели)			
0x06	UserApplicationName (имя приложения пользователя)			
0x07 .. 0x7F	Зарезервировано	Зависит от устройства		Extended
0x80 .. 0xFF	Приватные объекты, которые могут быть определены опционально. Этот диапазон зависит от продукта.			

Запрос:

Код функции	1 байт	0x2B
MEI Type	1 байт	0x0E
Read Device ID code	1 байт	01 / 02 / 03 / 04
Object Id	1 байт	0x00 или 0xFF

Ответ:

Код функции	1 байт	0x2B
MEI Type	1 байт	0x0E

Read Device ID code	1 байт	01 / 02 / 03 / 04
Conformity level	1 байт	0x01 или 0x02 или 0x03 или 0x81 или 0x82 или 0x83
Идет ли дальше что-то еще	1 байт	0x00 или 0xFF
Следующий Object Id	1 байт	Номер Object ID
Количество объектов	1 байт	
Список:		
Object ID	1 байт	
Длина объекта	1 байт	
Значение объекта	Длина объекта	Зависит от Object ID

Ошибка:

Код функции	1 байт	0xAB (результат 0x2B + 0x80)
Код исключения (exception code)	1 байт	01 или 02 или 03 или 04

Описание параметров запроса

MEI, назначенный на номер 14, идентифицирует запрос чтения идентификации (Read Identification Request).

Параметр "Read Device ID code" позволяет определить 4 типа доступа:

- 01: запрос для получения базовой идентификации устройства (поточковый доступ).
- 02: запрос для получения обычной идентификации устройства (поточковый доступ).
- 03: запрос для получения расширенной идентификации устройства (поточковый доступ).
- 04: запрос для получения одного из специфичных объектов идентификации (индивидуальный доступ).

Код исключения 03 будет отправлен обратно в ответе, если код Read device ID нелегальный.

В случае ответа, который не укладывается в одну транзакцию, нужно выполнить несколько транзакций (запрос/ответ). Байт Object Id дает идентификацию первого получаемого объекта. Для первой транзакции клиент должен установить Object Id в 0, чтобы получить начало идентификационных данных устройства. Для последующих транзакций клиент должен установить Object Id в значение, возвращаемое сервером в его предыдущем ответе.

Примечание: объект неделим, таким образом любой объект должен иметь размер, соответствующий размеру ответа транзакции.

Если Object Id не соответствует известному объекту, то сервер ответит так, как если бы было указание на объект 0 (перезапуск от начала).

В случае индивидуального доступа: ReadDevId code 04, Object Id в запросе дает идентификацию получаемого объекта, и если Object Id не соответствует любому известному объекту, то сервер возвратит ответ исключения с кодом исключения 02 (Illegal).

Описание параметров ответа

Function code: код функции 43 (десятичное) 0x2B (hex).

MEI Type: 14 (0x0E) тип MEI, назначенный для Device Identification Interface.

ReadDevId code: то же самое, как код запроса ReadDevId: 01, 02, 03 или 04.

Conformity Level: уровень соответствия идентификации устройства и тип поддерживаемого доступа.

- 0x01: базовая идентификация (только поточковый доступ).
- 0x02: обычная идентификация (только поточковый доступ).
- 0x03: расширенная идентификация (только поточковый доступ).
- 0x81: базовая идентификация (поточковый доступ и индивидуальный доступ).

0x82: обычная идентификация (поточковый доступ и индивидуальный доступ).

0x83: расширенная идентификация (поточковый доступ и индивидуальный доступ).

Что еще следует дальше: в случае кодов ReadDevId 01, 02 или 03 (поточковый доступ), если идентификационные данные не укладываются в одну транзакцию, то может потребоваться несколько транзакций запрос/ответ.

0x00: нет больше данных объекта.

0xFF: доступен другой объект идентификации, и требуется больше транзакций MODBUS.

В случае кода ReadDevId 04 (индивидуальный доступ) это поле должно быть установлено в 00.

Next Object Id: если "MoreFollows = FF", запрашивается идентификация следующего объекта. Если "MoreFollows = 00", то должно быть установлено в 00 (бесполезно).

Number Of Objects: количество объектов, возвращаемых в ответе (для индивидуального доступа Number Of Objects = 1).

Object0.Id: идентификация первого возвращаемого в PDU объекта (поточковый доступ) или запрошенный объект (индивидуальный доступ).

Object0.Length: длина первого объекта в байтах.

Object0.Value: значение первого объекта (Object0.Length байт).

...

ObjectN.Id: идентификация последнего объекта (в ответе).

ObjectN.Length: длина последнего объекта в байтах.

Пример запроса Read Device Identification для базовой идентификации устройства

В этом примере вся отправляемая информация помещается в один PDU ответа:

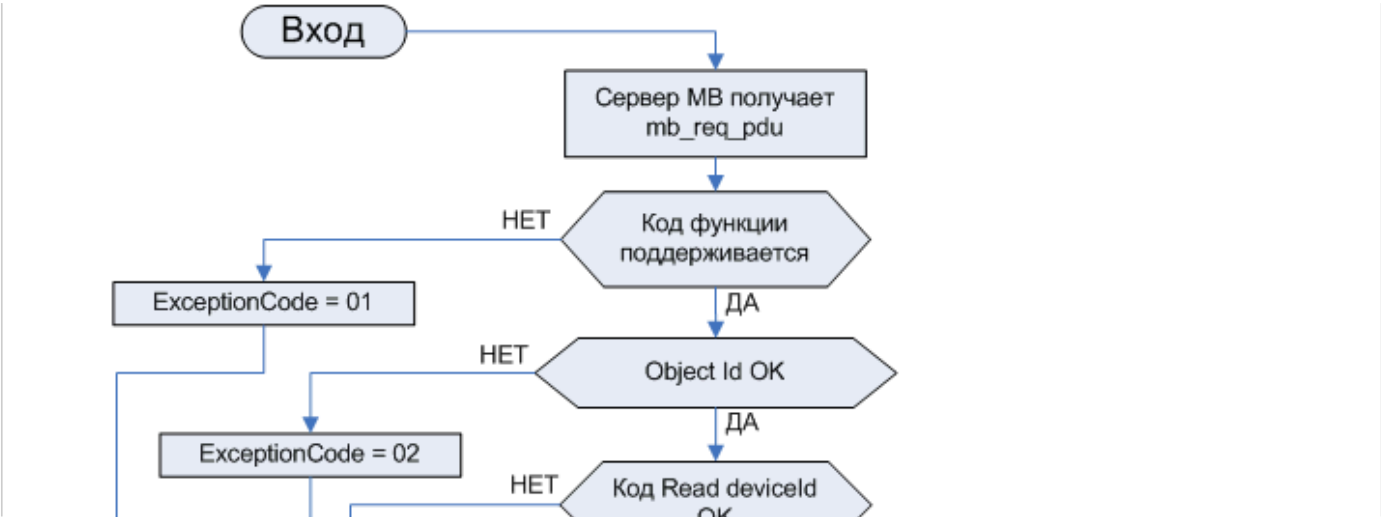
Запрос		Ответ	
Имя поля	HEX	Имя поля	HEX
Функция	28	Функция	28
MEI Type	0E	MEI Type	0E
Read Dev Id code	01	Read Dev Id Code	01
Object Id	00	Conformity Level	01
		Идет ли что-нибудь дальше	00
		NextObjectId	00
		Количество объектов	03
		Object Id	00
		Длина объекта	16
		Значение объекта	"Company identification"
		Object Id	01
		Длина объекта	0D
		Значение объекта	"Product code XX"
		Object Id	02
		Длина объекта	05
		Значение объекта	"V2.11"

В случае, когда устройство требует нескольких транзакций для отправки ответа, иницируются следующие транзакции.
Первая транзакция:

Запрос		Ответ	
Имя поля	HEX	Имя поля	HEX
Функция	28	Функция	28
MEI Type	0E	MEI Type	0E
Read Dev Id code	01	Read Dev Id Code	01
Object Id	00	Conformity Level	01
		Идет ли что-нибудь дальше	FF
		NextObjectId	02
		Количество объектов	03
		Object Id	00
		Длина объекта	16
		Значение объекта	"Company identification"
		Object Id	01
		Длина объекта	1C
		Значение объекта	"Product code XXXXXXXXXXXXXXXXXXXX"

Вторая транзакция:

Запрос		Ответ	
Имя поля	HEX	Имя поля	HEX
Функция	28	Функция	28
MEI Type	0E	MEI Type	0E
Read Dev Id code	01	Read Dev Id Code	01
Object Id	02	Conformity Level	01
		Идет ли что-нибудь дальше	00
		NextObjectId	00
		Количество объектов	03
		Object Id	02
		Длина объекта	05
		Значение объекта	"V2.11"



[7. Ответы исключений MODBUS]

Когда устройство-клиент (master) посылает запрос устройству-серверу (slave), от сервера ожидается нормальный ответ. Из-за запроса master может произойти одно из 4 событий:

- Если устройство-сервер приняло запрос без коммуникационной ошибки, то оно может обработать запрос нормальным образом, и вернуть нормальный ответ.
- Если сервер не принял запрос из-за коммуникационной ошибки, то ответ возвращен не будет. Программа клиента обработает таймаут для запроса.
- Если сервер принял запрос, но обнаружил коммуникационную ошибку (parity, LRC, CRC, ...), то ответ возвращен не будет. Программа клиента обработает таймаут для запроса.
- Если сервер принял запрос без коммуникационной ошибки, но не может обработать его (например, если был запрос на чтение несуществующего выхода или регистра), то сервер возвратит ответ исключения (exception response), информирующий клиента о природе ошибки.

Сообщение ответа исключения должно иметь два поля, которые отличают это сообщение от нормального ответа:

Поле кода функции. В нормальном ответе сервер как эхо посылает в ответе тот же код функции, который был в оригинальном запросе. Это все коды функции, у которых старший бит (MSB) в лог. 0 (их значения меньше 0x80 hex). В ответе исключения (exception response) сервер установит в лог. 1 MSB кода функции. Это делает код функции в ответе исключения однозначно больше на 0x80 (hex), чем значение кода функции в нормальном ответе.

По установленному MSB в коде функции программа приложения клиента может распознать ответ исключения, и проверить поле данных для кода исключения.

Поле данных. В нормальном ответе сервер может вернуть данные или статистику (любую информацию, которую требовал запрос). В ответе исключения сервер вернет в поле данных код исключения (exception code). Это определит состояние сервера, которое вызвало исключение.

Пример запроса клиента и ответа исключения сервера

Запрос		Ответ	
Имя поля	HEX	Имя поля	HEX
Функция	01	Функция	81
Старший байт начального адреса	04	Код исключения	02
Младший байт начального адреса	A1		
Старший байт количества выходов	00		
Младший байт количества выходов	01		

В этом примере клиент адресует запрос устройству сервера. Код функции (01) предназначен для операции Read Output Status. Он запрашивает статус выхода по адресу 1185 (0x04A1 hex). Обратите внимание, что считывается только один выход, как указано

в поле количества выходов (0001).

Если такой адрес выхода не существует в устройстве сервера, то сервер вернет ответ исключения (exception response) с

Список кодов исключений (MODBUS Exception Codes, коды указаны в HEX-формате) приведен в следующей таблице.

Код	Имя	Что означает
01	ILLEGAL FUNCTION	Код функции, полученный в запросе, задает недопустимое действие для сервера (т. е. для подчиненного устройства MODBUS). Это может быть из-за того, что этот код функции применим только для более новых устройств, и он не реализован в выбранном устройстве. Также это может показывать, что сервер находится в неподходящем состоянии для обработки запроса этого типа, например из-за того, что он не сконфигурирован, и при этом с него запрашиваются значения регистров.
02	ILLEGAL DATA ADDRESS	Адрес данных, принятый в запросе, недопустим для сервера. Другая специфическая ситуация - комбинация индекса (reference number) и количество данных в транзакции (transfer length) недопустимы. Для контроллера со 100 регистрами PDU адресует первый регистр как 0, и последний как 99. Если был выдан запрос, когда начальный адрес регистра 96, и количество запрашиваемых регистров 4, то этот запрос будет успешно выполнен (как минимум по соответствию адресов) для регистров 96, 97, 98, 99. Если запрос был выдан с начальным адресом 96, но количество регистров было задано 5, то этот запрос потерпит неудачу с кодом исключения 0x02 "Illegal Data Address", потому что была сделана попытка выйти за пределы рабочих адресов регистров - потребовалось прочитать регистры 96, 97, 98, 99 и 100, но регистр с адресом 100 не существует.
03	ILLEGAL DATA VALUE	Значение, содержащееся в поле данных запроса, имеет недопустимое значение для сервера. Это показывает ошибку в остальной части структуры сложного запроса, например неправильную подразумеваемую длину. Это конкретно НЕ означает, что элемент данных, представленный для хранения в регистре, имеет значение, не соответствующее ожиданиям прикладной программы, поскольку протокол MODBUS не знает о смысле и значимости какого-либо конкретного значения какого-либо конкретного регистра.
04	SLAVE DEVICE FAILURE	Произошла невосстановимая ошибка, когда сервер попытался выполнить запрошенное действие.
05	ACKNOWLEDGE	Специализированное использование вместе с командами программирования. Сервер (подчиненное устройство) должен принять запрос и обработать его, но для этого требуется большой интервал времени. Этот ответ будет возвращен, чтобы предотвратить ошибку таймаута в клиенте (главном устройстве). Клиент может выдать следующее сообщение Poll Program Complete, чтобы определить, завершена ли обработка.
06	SLAVE DEVICE BUSY	Специализированное использование вместе с командами программирования. Сервер (подчиненное устройство) занимается обработкой длительной команды программы. Клиент (главное устройство) должно повторно передать сообщение позже, когда сервер освободится.
08	MEMORY PARITY ERROR	Специализированное использование вместе с кодами функции 20 и 21 и ссылочным типом 6. Показывает, что расширенная область файла не прошла проверку целостности. Сервер попытался прочитать элемент данных в файле, но в памяти детектирована ошибка четности. Клиент (главное устройство) может повторить запрос, однако возможно понадобится сервисное обслуживание (ремонт) сервера.
0A	GATEWAY PATH UNAVAILABLE	Специализированное использование вместе со шлюзами. Показывает, что шлюз не может выделить внутренний путь от входного коммуникационный путь от входного порта до выходного порта, что требовалось для обработки запроса. Обычно это означает, что шлюз неправильно сконфигурирован или перегружен.
0B	GATEWAY TARGET DEVICE FAILED TO RESPOND	Специализированное использование вместе со шлюзами. Показывает, что от целевого устройства не был получен ответ. Обычно означает, что устройство отсутствует в сети.

[Приложение A: зарезервированные коды функций MODBUS, субкоды и типы MEI]

Следующие коды функций и субкоды функций не входят в публичную спецификацию, и эти коды и субкоды функции специально зарезервированы. Формат представляет собой код/субкод или просто код функции, где зарезервированы следующие комбинации: 8 / 19; 8 / 21 .. 65535, 9, 10, 13, 14, 41, 42, 90, 91, 125, 126 и 127.

Эта спецификация в настоящий момент предоставляет код функции 43 и его MEI Type 14 для Device Identification и MEI Type 13 для CANopen General Reference Request и PDU ответа в качестве транспортов инкапсулированного интерфейса (Encapsulated Interface Transports).

Следующие коды функции и типы MEI не должны быть частью этой публичной спецификации, и эти коды функции и типы MEI специально зарезервированы: 43 / 0 .. 12 и 43 / 15 .. 255. В этой спецификации код функции пользователя (User Defined Function) дает тот же или подобный результат, как не поддерживаемый Encapsulated Interface Transport.

MODBUS является зарегистрированной торговой маркой Schneider Automation Inc.

[Словарик]

ADU Application Data Unit.

HDLC High level Data Link Control.

HMI Human Machine Interface.

IETF Internet Engineering Task Force.

I/O Input/Output, ввод/вывод.

IP Internet Protocol.

LSB Least Significant Bit, младший значащий бит.

MAC Medium Access Control.

MB MODBUS Protocol.

MBAP MODBUS Application Protocol.

MEI MODBUS Encapsulated Interface, дополнительный код функции, передаваемый за основным. Стандарт определяет MEI 13 (0x0D), предназначенный для инкапсуляции протокола CANopen. MEI 14 (0x0E) используется для получения информации об устройстве и MEI в диапазонах 0—12 и 15—255 зарезервированы.

MSB Most Significant Bit, старший значащий бит.

OSI Open Systems Interconnection, открытая модель организации обмена данными через сеть.

PDU Protocol Data Unit, элемент обмена данными.

PLC Programmable Logic Controller, программируемый логический контроллер.

RFC Request for Comments, рабочее предложение - документ из серии пронумерованных информационных документов Интернета, содержащих технические спецификации и стандарты, широко применяемые во всемирной сети (из Википедии).

TCP Transport Control Protocol.

клиент главное устройство (master) сети MODBUS, которое отправляет запросы/команды серверам MODBUS и получает от них ответы.

сервер подчиненное устройство (slave) сети MODBUS, которое принимает запросы/команды клиента MODBUS, выполняет их и отправляет соответствующие ответы клиенту MODBUS.

[Ссылки]

1. MODBUS APPLICATION PROTOCOL SPECIFICATION V1.1b site:modbus.org.
2. MODBUS Messaging Implementation Guide V1.0a site:modbus.org.
3. RFC 791, Internet Protocol, Sep81 DARPA.
4. [Порядок следования байт \(endianness\)](#).
5. [Библиотека FreeMODBUS](#).